



รายงานโครงงานฉบับสมบูรณ์

เรื่อง ระบบแจ้งเตือนคิวรับอาหารและเครื่องดื่มผ่านเว็บ Food Queue Notifier

(Food Queue Notifier: A Web-Based Food and Beverage Queue Notification System)

โดย

นายกานต์ดิษฐ์ นามสุดตา (673380392-1)

นายอนันต์เอกก์ ใหญ่พงศกร (673380430-9)

นายคมชาญ วรสาร (673380396-3)

นายธีรัช ลิ้มประยูรวงศ์ (673380408-2)

เสนอ

รศ. ดร.ปัญญาพล หอระตะ

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CP353002

Principles of Software Design and Development

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

ภาคการศึกษาที่ 1 ปีการศึกษา 2569



รายงานโครงงานฉบับสมบูรณ์

เรื่อง ระบบแจ้งเตือนคิวรับอาหารและเครื่องดื่มผ่านเว็บ Food Queue Notifier

(Food Queue Notifier: A Web-Based Food and Beverage Queue Notification System)

โดย

นายกานดิทัต นามสุดตา (673380392-1)

นายอนันต์เอกก์ ใหญ่พงศกร (673380430-9)

นายคมชาญ วรสาร (673380396-3)

นายธีรัช ลิ้มประยูรวงศ์ (673380408-2)

เสนอ

รศ. ดร.ปัญญาพล หอระตะ

รายงานนี้เป็นส่วนหนึ่งของรายวิชา CP353002

Principles of Software Design and Development

สาขาวิชาวิทยาการคอมพิวเตอร์ วิทยาลัยการคอมพิวเตอร์

มหาวิทยาลัยขอนแก่น

ภาคการศึกษาที่ 1 ปีการศึกษา 2569

บทคัดย่อ

ร้านอาหารและเครื่องดื่มที่มีลูกค้าจำนวนมากในช่วงเร่งด่วนมักประสบปัญหาคิวรับสินค้าแออัด เพราะลูกค้าไม่ทราบว่าจะออเดอร์ของตนพร้อมรับเมื่อใดโครงการนี้จึงมีวัตถุประสงค์เพื่อพัฒนาระบบ Food Queue Notifier ซึ่งเป็นระบบสั่งอาหาร จัดการคิว และแจ้งเตือนสถานะคิวอัตโนมัติผ่าน Browser Push เพื่อประยุกต์ใช้ Design Pattern แบบ State, Observer และ Strategy เพื่อออกแบบความปลอดภัยของระบบ และเพื่อทดสอบการทำงานด้วยชุดทดสอบอัตโนมัติ โดยมีกลุ่มเป้าหมายคือลูกค้าผู้รับบริการและพนักงานของร้าน

ระบบพัฒนาด้วย Spring Boot 3.2.5 ภาษา Java ฐานข้อมูล PostgreSQL 16 ร่วมกับ Flyway หน้าเว็บใช้ Thymeleaf และ JavaScript และใช้ Service Worker กับ VAPID สำหรับ Web Push ลูกค้าไม่ต้องสมัครบัญชีและใช้ owner token เฉพาะออเดอร์ ส่วนพนักงานเข้าสู่ระบบด้วยรหัสผ่านที่แฮชด้วย BCrypt และระบบป้องกัน CSRF การเปลี่ยนสถานะคิวใช้ State Pattern เผยแพร่เหตุการณ์ให้ Observer ส่งแจ้งเตือนหลัง transaction commit และเลือกช่องทางส่งด้วย Strategy Pattern

ผลการดำเนินงานพบว่าระบบทำงานครบตามขอบเขต คือ สั่งอาหาร รับเลขคิวรายวัน ติดตามสถานะ แก้ไขหรือยกเลิกตามกฎของสถานะ จัดการเมนูและคิวฝั่งพนักงาน และแจ้งเตือนเมื่ออาหารพร้อม ชุดทดสอบ Java ผ่าน 285 tests และชุดทดสอบ JavaScript ผ่าน 14 ไฟล์ นอกจากนี้ได้ deploy ระบบบน Render ร่วมกับ Neon PostgreSQL และผู้ทดสอบยืนยันว่าสั่งอาหารและรับการแจ้งเตือนที่แสดงเลขคิวถูกต้องได้หนึ่งรอบ

ข้อจำกัดของโครงการคือ ยังไม่ได้ทำ acceptance ครบทุกรายการและยังไม่มีผลทดสอบบนอุปกรณ์จริงหลายรุ่น การส่งแจ้งเตือนไม่มี retry อัตโนมัติและไม่มี outbox และคำขอเลื่อนสถานะซ้ำยังไม่มีกลไก idempotency ข้อเสนอแนะคือควรเพิ่มกลไกส่งซ้ำ เพิ่มการป้องกันคำขอซ้ำ ทดสอบบนอุปกรณ์จริง และพิจารณาช่องทางแจ้งเตือนอื่นก่อนนำไปใช้งานจริง

คำสำคัญ: ระบบจัดการคิว, การแจ้งเตือน, Web Push, Design Pattern, Spring Boot

สารบัญ

บทคัดย่อ	ก
สารบัญ	ค
สารบัญตาราง	ง
สารบัญภาพ	จ
บทที่ 1 ความเป็นมาและความสำคัญของปัญหา	1
1.1 ความเป็นมาและความสำคัญของปัญหา	1
1.2 วัตถุประสงค์ของโครงการ	1
1.3 ขอบเขตของโครงการ	2
1.4 ผลที่จะได้รับ	2
1.5 ผู้รับผิดชอบโครงการ	3
1.6 นิยามศัพท์เฉพาะ	3
บทที่ 2 แนวคิดทฤษฎีและงานที่เกี่ยวข้อง	5
2.1 ระบบจัดการคิว	5
2.2 รูปแบบการออกแบบและหลักการ SOLID	5
2.3 Spring Boot และ Spring Data JPA	6
2.4 PostgreSQL และ Flyway	6
2.5 ความปลอดภัยของเว็บแอปพลิเคชัน	7
2.6 Web Push และ VAPID	7
2.7 การทดสอบซอฟต์แวร์และเครื่องมือสนับสนุน	7
2.8 งานที่เกี่ยวข้อง	8
บทที่ 3 วิธีการดำเนินงานและการวิเคราะห์ออกแบบระบบ ...	9
3.1 กระบวนการดำเนินงานและเครื่องมือ	9

3.2 สถาปัตยกรรมของระบบ	9
3.3 ผู้ใช้และกรณีการใช้งาน	10
3.4 การออกแบบฐานข้อมูล	12
3.5 การออกแบบสถานะของคิว	15
3.6 การออกแบบด้วย Design Pattern และหลักการ SOLID ...	16
3.6.1 Design Pattern กลุ่ม Behavioral	16
3.6.2 หลักการ SOLID	21
3.7 การออกแบบความปลอดภัย	23
3.8 กระบวนการทำงานหลัก	24
3.9 การติดตั้งและการใช้งาน	30
3.10 วิธีการทดสอบ	32
บทที่ 4 ผลการดำเนินงาน	34
4.1 สภาพแวดล้อมและเครื่องมือ	34
4.2 ผลการพัฒนาระบบ (วัตถุประสงค์ข้อ 1)	34
4.3 ผลการประยุกต์ Design Pattern (วัตถุประสงค์ข้อ 2) ...	35
4.4 ผลการออกแบบความปลอดภัย (วัตถุประสงค์ข้อ 3)	36
4.5 ผลการทดสอบอัตโนมัติ (วัตถุประสงค์ข้อ 4)	37
4.6 ผลการทดสอบบนระบบที่ deploy	38
4.7 สิ่งที่ยังไม่ยืนยัน	39
บทที่ 5 สรุปผล อภิปรายผลและข้อเสนอแนะ	40
5.1 สรุปผลการดำเนินงาน	40
5.2 อภิปรายผล	40
5.3 ปัญหาและอุปสรรค	41
5.4 ข้อจำกัดของระบบ	41

5.5 ข้อเสนอแนะ	41
เอกสารอ้างอิง	43
ภาคผนวก	44
ภาคผนวก ก โครงสร้างโพลเดอร์ของโปรเจกต์	44
ภาคผนวก ข ภาพหน้าจอและหลักฐานการทดสอบเพิ่มเติม ...	44

สารบัญตาราง

ตารางที่ 1.1 สมาชิกและส่วนรับผิดชอบ	3
ตารางที่ 3.1 ผู้ใช้และกรณีการใช้งานหลัก	11
ตารางที่ 3.2 ตารางฐานข้อมูลหลักและข้อจำกัดสำคัญ	13
ตารางที่ 3.3 กฎการเปลี่ยนสถานะและการแก้ไขตามสถานะ ...	15
ตารางที่ 3.4 หลักการ SOLID และตัวอย่างในระบบ	22
ตารางที่ 3.5 สิทธิ์การเข้าถึงตามกลุ่มเส้นทาง	23
ตารางที่ 4.1 ผลการพัฒนาตามฟังก์ชัน	34
ตารางที่ 4.2 Design Pattern ที่ใช้และชุดทดสอบที่เกี่ยวข้อง ...	35
ตารางที่ 4.3 กลไกความปลอดภัยและชุดทดสอบที่เกี่ยวข้อง ...	36
ตารางที่ 4.4 ขอบเขตของชุดทดสอบอัตโนมัติ	37
ตารางที่ 4.5 ผลการทดสอบบน Neon และ Render	38
ตารางที่ ก.1 โครงสร้างโฟลเดอร์ของโปรเจกต์	44

สารบัญภาพ

ภาพที่ 3.1 ภาพรวมส่วนประกอบของระบบ (Component Overview) ...	10
ภาพที่ 3.2 Use case ของระบบ	12
ภาพที่ 3.3 ER Diagram ของระบบ	14
ภาพที่ 3.4 State diagram ของคิว	16
ภาพที่ 3.5 Class diagram ของ Layered Architecture และ MVC ...	18
ภาพที่ 3.6 Class diagram ของ State Pattern	19
ภาพที่ 3.7 Class diagram ของ Observer Pattern	20
ภาพที่ 3.8 Class diagram ของ Strategy Pattern	21
ภาพที่ 3.9 Sequence diagram การสร้างออเดอร์	25
ภาพที่ 3.10 Sequence diagram การสมัครและยกเลิก Push ของออเดอร์ ...	27
ภาพที่ 3.11 Sequence diagram การแจ้งเตือนหลัง commit เมื่อคิวเป็น READY ...	29
ภาพที่ 3.12 การรันในเครื่องที่ใช้ทดสอบ	30
ภาพที่ 3.13 CI บน GitHub Actions	31
ภาพที่ 3.14 การ deploy บน Render และ Neon PostgreSQL ...	32
ภาพที่ 4.1 หน้าเมนูและตะกร้าสินค้า	34
ภาพที่ 4.2 หน้าติดตามคิวของลูกค้า	35
ภาพที่ 4.3 หน้าจอพนักงานสำหรับเลื่อนสถานะคิว	35
ภาพที่ 4.4 การแจ้งเตือนเมื่ออาหารพร้อมบนอุปกรณ์	35

บทที่ 1 ความเป็นมาและความสำคัญของปัญหา

1.1 ความเป็นมาและความสำคัญของปัญหา

ร้านอาหารและเครื่องดื่มที่มีลูกค้าจำนวนมากในช่วงเวลาเร่งด่วนมักประสบปัญหาคิวรับสินค้า แออัด ลูกค้าต้องยืนรอนำร้านเพราะไม่ทราบว่ามีออเดอร์ของตนพร้อมรับเมื่อใด ส่วนร้านค้าก็ต้องเรียกคิวด้วยเสียงหรือป้ายหน้าร้าน ซึ่งบางครั้งลูกค้าไม่ได้ยินจนคิวตกหล่นหรืออาหารเย็นชืด

ผู้พัฒนาจึงจัดทำระบบ **Food Queue Notifier** ซึ่งเป็นระบบแจ้งเตือนคิวรับอาหารและเครื่องดื่ม ลูกค้าสั่งเมนูผ่านหน้าเว็บ รับเลขคิวประจำวัน ติดตามสถานะคิวได้ด้วยตนเอง และได้รับการแจ้งเตือนผ่านเบราว์เซอร์ (Browser Push) เมื่อออเดอร์พร้อมรับ ลูกค้าจึงไม่จำเป็นต้องรอนำร้านตลอดเวลา ขณะที่พนักงานจัดการสถานะคิวผ่านหน้าจอพนักงานได้อย่างเป็นระบบ โครงการนี้เป็นส่วนหนึ่งของรายวิชา CP353002 Principles of Software Design and Development จึงเน้นการประยุกต์ Design Pattern และหลักการออกแบบซอฟต์แวร์เข้ากับระบบที่ใช้งานได้จริง

1.2 วัตถุประสงค์ของโครงการ

- 1) เพื่อพัฒนาระบบสั่งอาหาร จัดการคิว และแจ้งเตือนสถานะคิวอัตโนมัติ
- 2) เพื่อประยุกต์ใช้ Design Pattern ได้แก่ State, Observer และ Strategy ในการออกแบบระบบ
- 3) เพื่อออกแบบระบบให้มีความปลอดภัยในการยืนยันตัวตน การกำหนดสิทธิ์ และการป้องกันการโจมตีแบบ CSRF
- 4) เพื่อทดสอบการทำงานของระบบด้วยชุดทดสอบอัตโนมัติและการทดสอบบนระบบที่ deploy

1.3 ขอบเขตของโครงการ

ด้านการใช้งาน ระบบรองรับผู้ใช้สองกลุ่ม คือ ลูกค้าและพนักงาน (STAFF) ลูกค้าไม่ต้องสมัครบัญชี โดยระบบออก token เฉพาะออเดอร์ให้เก็บในเบราว์เซอร์ ลูกค้าสามารถดูเมนู ใส่ตะกร้า สร้างออเดอร์ ติดตามคิวและดูประวัติ แก้ออเดอร์เมื่อ WAITING ยกเลิกเมื่อ WAITING หรือ PREPARING และเปิดหรือปิดการแจ้งเตือนของแต่ละออเดอร์ ส่วนพนักงานเข้าสู่ระบบ จัดการเมนูและรูป ดูรายการออเดอร์ เลื่อนสถานะคิว ดูบันทึกการส่งแจ้งเตือน และสร้าง QR หน้าเมนู

ด้านเทคโนโลยี พัฒนาด้วย Spring Boot 3.2.5 (MVC, Security, Data JPA) ภาษา Java (build ด้วย JDK 21 และ compile เป้าหมาย Java 17) Thymeleaf และ JavaScript ผังเบราว์เซอร์

จัดการโปรเจกต์ด้วย Maven ใช้ฐานข้อมูล PostgreSQL 16 พร้อม Flyway ใช้ Service Worker และ VAPID สำหรับ Browser Push และ deploy บน Render ร่วมกับ Neon PostgreSQL

ขอบเขตที่ไม่รวม ระบบไม่มีขั้นตอนชำระเงินใน flow ปัจจุบัน ไม่มีบัญชีลูกค้าหรือการกู้ token และผลการแสดง Push บนอุปกรณ์จริงยังไม่ได้รับการยืนยันครบทุกอุปกรณ์

1.4 ผลที่จะได้รับ

ลูกค้าประหยัดเวลารอและทราบสถานะออเดอร์ได้ทันเวลา ร้านค้าบริหารคิวได้เป็นระบบ ลดการเรียกคิวผิดพลาดและความแออัดหน้าร้าน ส่วนผู้พัฒนาได้ประยุกต์ความรู้ด้านการออกแบบซอฟต์แวร์ การทำงานเป็นทีมผ่าน Git workflow (Pull Request และการรีวิวกัด) และการทดสอบอัตโนมัติ

1.5 ผู้รับผิดชอบโครงการ

ตารางที่ 1.1 สมาชิกและส่วนรับผิดชอบ

สมาชิก	ส่วนรับผิดชอบ
นายกานติทัต นามสุดตา	หน้าเว็บ ตะกร้า ประวัติ/ติดตามคิว หน้าจอพนักงาน QR และ runtime/CI
นายอนันต์เอกก์ ใหญ่พงศกร	schema/migration, persistence, เมนู/ภาพเมนู, ออเดอร์ เลขคิวรายวัน และ OpenAPI
นายคมชาญ วรสาร	State, เปลี่ยนสถานะ/ยกเลิกคิว, token, สิทธิ์ออเดอร์, Security/CSRF และสิทธิ์ Swagger
นายจีรัช ลิ้มประยูรวงศ์	Subscription, Observer, Strategy/sender, delivery, log และ Browser Push

ทีมใช้ branch main สำหรับรุ่นส่งมอบ develop สำหรับรวมงาน และให้สมาชิกทำงานบน branch ส่วนตัว รวมงานผ่าน Pull Request ที่มีสมาชิกอีกคนรีวิว [ใส่ตารางระยะเวลาดำเนินงานตามจริง]

1.6 นิยามศัพท์เฉพาะ

Browser Push คือการส่งการแจ้งเตือนไปยังเบราว์เซอร์ของผู้ใช้ **VAPID** คือมาตรฐานระบุตัวผู้ส่ง Push **STAFF** คือผู้ใช้ฝั่งพนักงานร้าน **CSRF** คือการโจมตีโดยหลอกให้ผู้ใช้ส่งคำขอโดยไม่รู้ตัว

Flyway คือเครื่องมือจัดการ migration ของฐานข้อมูล **Owner token** คือรหัสเฉพาะของออเดอร์ที่ใช้พิสูจน์ความเป็นเจ้าของออเดอร์โดยไม่ต้องมีบัญชี

บทที่ 2 แนวคิดทฤษฎีและงานที่เกี่ยวข้อง

2.1 ระบบจัดการคิว

ระบบจัดการคิวคือระบบที่จัดลำดับผู้รับบริการตามลำดับก่อนหลังและแสดงสถานะการรอ ซึ่งช่วยลดความสับสนและความแออัดของผู้ใช้บริการ ระบบคิวแบบดิจิทัลอย่าง QueQ ให้ผู้ใช้รับหมายเลขคิวผ่านสมาร์ทโฟนและได้รับการแจ้งเตือนเมื่อใกล้ถึงคิว ซึ่งเป็นแนวคิดเดียวกับระบบของโครงการนี้ แต่โครงการนี้นั้นเน้นการออกแบบซอฟต์แวร์เชิงวัตถุและการแจ้งเตือนผ่านเบราว์เซอร์โดยไม่ต้องติดตั้งแอปพลิเคชันเพิ่ม

2.2 รูปแบบการออกแบบและหลักการ SOLID

Design Pattern คือแนวทางแก้ปัญหาการออกแบบซอฟต์แวร์ที่พบบ่อยและนำกลับมาใช้ซ้ำได้ (Gamma et al., 1994) โครงการนี้เลือกใช้ GoF กลุ่ม Behavioral จำนวน 3 แบบ ดังนี้

State Pattern ใช้เมื่อพฤติกรรมของออบเจกต์ขึ้นอยู่กับสถานะปัจจุบัน ระบบนี้นำมาใช้กับสถานะของคิว 5 สถานะ ได้แก่ WAITING, PREPARING, READY, COMPLETED และ CANCELLED เพื่อควบคุมว่าสถานะใดเปลี่ยนไปสถานะใดได้ จึงลดเงื่อนไข if-else ที่กระจัดกระจาย

Observer Pattern ใช้เมื่อต้องการให้ออบเจกต์หนึ่งแจ้งผู้สนใจเมื่อสถานะเปลี่ยน ระบบนี้ใช้กับการเปลี่ยนสถานะคิวเป็นตัวกระตุ้น และให้ผู้สังเกตการณ์ทำหน้าที่ส่งแจ้งเตือนหลังบันทึกข้อมูลสำเร็จ (AFTER_COMMIT) เพื่อไม่ให้ส่งแจ้งเตือนในกรณีที่ธุรกรรมล้มเหลว

Strategy Pattern ใช้เมื่อต้องการสลับอัลกอริทึมได้โดยไม่แก้ไขโค้ดส่วนที่เรียกใช้ ระบบนี้ใช้กับช่องทางส่งแจ้งเตือน โดยมีโหมด console สำหรับ preview และโหมด webpush สำหรับส่งจริง ทำให้สลับวิธีส่งได้ด้วยการตั้งค่า

นอกจากนี้ระบบใช้รูปแบบเชิงสถาปัตยกรรม ได้แก่ Layered Architecture, MVC, Repository, Service Layer, DTO และ Mapper รวมถึง Dependency Injection และอธิบายการออกแบบด้วยหลักการ SOLID ได้แก่ Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation และ Dependency Inversion (Martin, 2003)

2.3 Spring Boot และ Spring Data JPA

Spring Boot เป็นเฟรมเวิร์กภาษา Java ที่ช่วยลดการตั้งค่าและรวมองค์ประกอบสำหรับสร้างเว็บแอปพลิเคชันและ REST API ส่วน JPA/Hibernate ใช้แปลงข้อมูลระหว่างออบเจกต์กับตารางฐานข้อมูล ระบบนี้ตั้งค่า `ddl-auto: validate` เพื่อให้ Hibernate ตรวจสอบว่า schema ตรงกับ entity โดยไม่แก้ฐานข้อมูลเอง หน้าเว็บใช้ Thymeleaf เป็น template engine และใช้ springdoc-openapi สร้างเอกสาร API ผ่าน Swagger UI

2.4 PostgreSQL และ Flyway

PostgreSQL เป็นระบบจัดการฐานข้อมูลเชิงสัมพันธ์แบบโอเพนซอร์ส ระบบใช้ Flyway จัดการ migration แบบมีเวอร์ชัน (V1-V6) เพื่อให้โครงสร้างฐานข้อมูลของสมาชิกทุกคนตรงกันและตรวจสอบย้อนหลังได้

2.5 ความปลอดภัยของเว็บแอปพลิเคชัน

ระบบใช้ BCrypt แฮชรหัสผ่านพนักงาน ซึ่งออกแบบให้ช้าโดยตั้งใจเพื่อต้านการเดารหัส ใช้ **owner token** ระบุสิทธิ์เข้าถึงออเดอร์ของลูกค้าแต่ละราย ควบคุมสิทธิ์ตามบทบาท (เจ้าของออเดอร์ /STAFF) และป้องกัน CSRF ด้วย Spring Security โดยตรวจโทเค็นในคำขอที่เปลี่ยนแปลงข้อมูล (OWASP, n.d.) นอกจากนี้ใช้การล็อกแถวแบบ pessimistic write เพื่อควบคุมการแก้ไขพร้อมกัน

2.6 Web Push และ VAPID

Web Push ช่วยให้เว็บส่งการแจ้งเตือนถึงผู้ใช้ได้แม้ไม่ได้เปิดหน้าเว็บ โดยเบราว์เซอร์ลงทะเบียน subscription ผ่าน Service Worker กับบริการ Push ของผู้ให้บริการเบราว์เซอร์ (Thomson et al., 2016) จากนั้นเซิร์ฟเวอร์ส่งข้อความโดยระบุตัวตนด้วยคีย์คู่กุญแจ VAPID (Thomson & Beverloo, 2017) สถานะการส่งของระบบแยกเป็น PREVIEW (ไม่ส่งไป provider), ACCEPTED (provider รับคำขอแล้ว แต่ไม่ยืนยันว่าอุปกรณ์แสดงผลจริง) และ FAILED

2.7 การทดสอบซอฟต์แวร์และเครื่องมือสนับสนุน

ระบบใช้ชุดทดสอบ Java (unit, MVC/Security และ PostgreSQL integration tests บนฐานข้อมูลแบบ embedded) และชุดทดสอบ JavaScript ด้วย Node.js สำหรับ UI และ Service Worker รัน build ผ่าน Maven (`mvn clean verify`) และใช้ GitHub Actions เป็น CI ส่วนการแพ็คเกจและรันใช้ Docker Compose

2.8 งานที่เกี่ยวข้อง

[เพิ่มงานที่เกี่ยวข้อง 2-3 รายการ เช่น ระบบคิวดิจิทัลที่มีให้บริการ หรือบทความเกี่ยวกับ Web Push พร้อมแหล่งอ้างอิงตามรูปแบบที่อาจารย์กำหนด]

บทที่ 3 วิธีการดำเนินงานและการวิเคราะห์ออกแบบระบบ

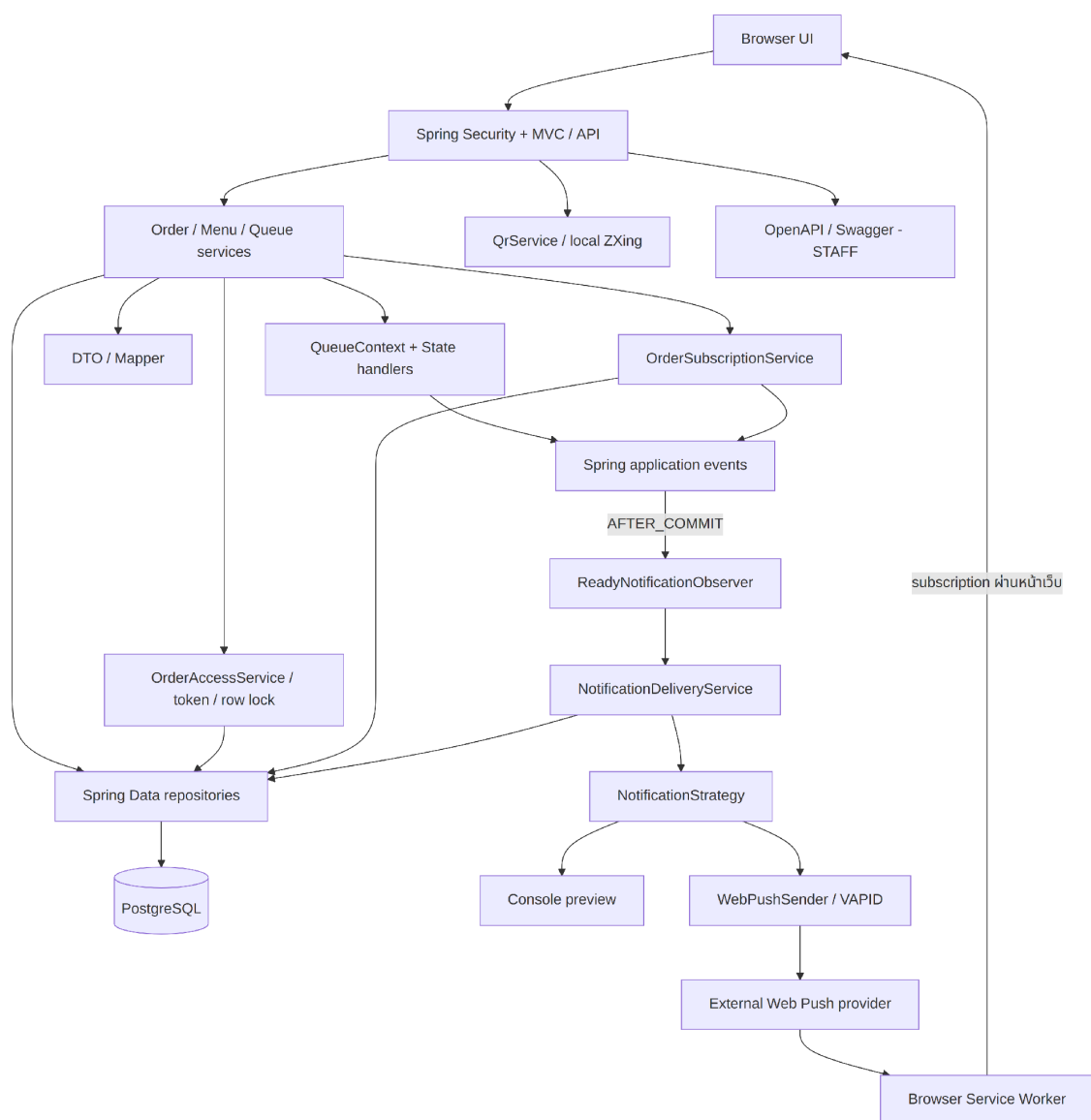
3.1 กระบวนการดำเนินงานและเครื่องมือ

โครงการนี้เป็นงานพัฒนาซอฟต์แวร์ ผู้พัฒนาดำเนินงานตามขั้นตอนต่อไปนี้ คือ วิเคราะห์ปัญหา และกำหนดกลุ่มผู้ใช้ ออกแบบสถาปัตยกรรม ฐานข้อมูล สถานะของคิว Design Pattern และความปลอดภัย พัฒนาระบบ ทดสอบด้วยชุดทดสอบอัตโนมัติ และ deploy เพื่อทดสอบบนระบบจริง กลุ่มเป้าหมายของระบบคือลูกค้าผู้รับบริการและพนักงานของร้านอาหารหรือเครื่องดื่มที่มีคิวรับสินค้า

ทีมแบ่งงานตามส่วนรับผิดชอบในตารางที่ 1.1 ใช้ Git บน GitHub โดยมี branch main สำหรับรุ่นส่งมอบ develop สำหรับรวมงาน และ branch ส่วนตัวของสมาชิก รวมงานผ่าน Pull Request ที่มีสมาชิกอีกคนรีวิว ใช้ GitHub Actions เป็น CI รัน JavaScript tests และ mvn clean verify กับ PostgreSQL แบบแยก และใช้ Docker Compose สำหรับรันฐานข้อมูล และแอปในเครื่อง เนื่องจากโครงการเป็นการพัฒนาซอฟต์แวร์ ผลการดำเนินงานจึงประเมินจากผลการทดสอบการทำงานของระบบ

3.2 สถาปัตยกรรมของระบบ

ระบบออกแบบเป็นสถาปัตยกรรมแบบชั้น (Layered Architecture) คำขอจากเบราว์เซอร์ผ่าน Spring Security เข้าสู่ Controller จากนั้นส่งต่อ Service ซึ่งเรียก Repository เพื่อเข้าถึง PostgreSQL ข้อมูลที่ส่งออกจากทาง API ผ่าน DTO และ Mapper จึงไม่เปิดเผย entity ทั้งก่อนหรือ token hash การแยกหน้าที่นี้ทำให้กฎธุรกิจไม่ผูกกับ HTTP และการใช้ Dependency Injection ผ่าน constructor ทำให้ทดสอบโดยแทน dependency ได้ ส่วนหน้าเว็บใช้ Thymeleaf เป็น template และ JavaScript ฝั่งเบราว์เซอร์เรียก API เพื่อโหลดเมนู คิว และสถานะ ดังภาพที่ 3.1



ภาพที่ 3.1 ภาพรวมส่วนประกอบของระบบ (Component Overview)

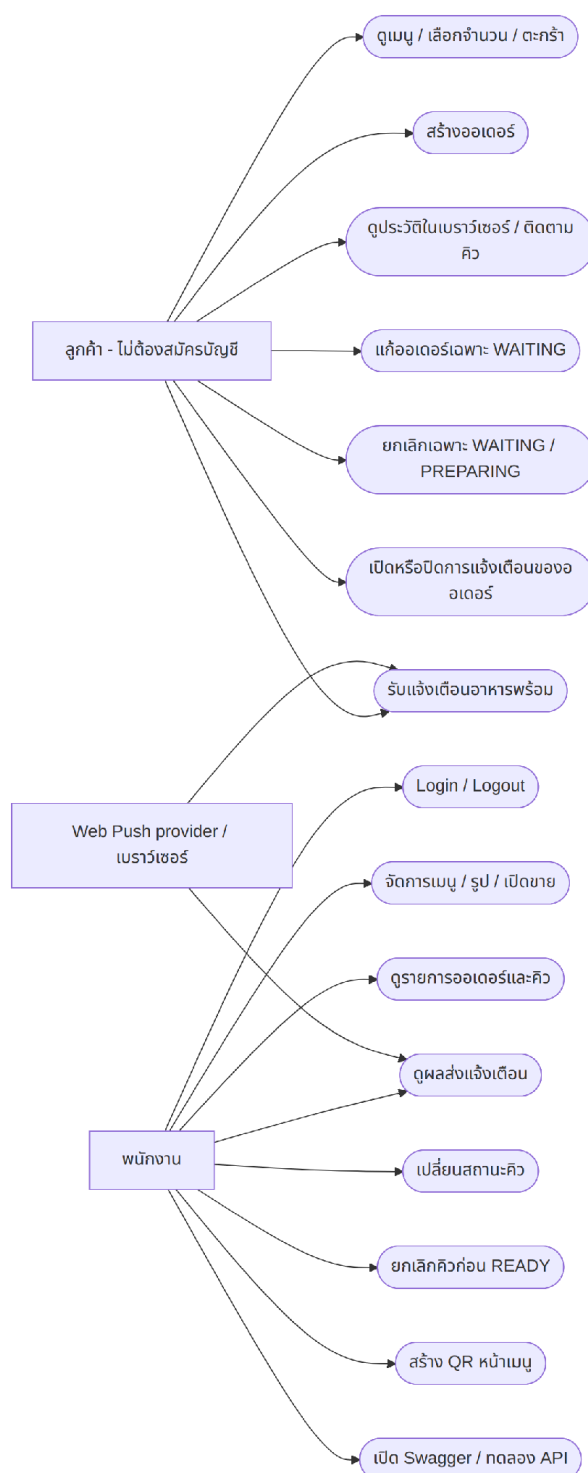
3.3 ผู้ใช้และกรณีการใช้งาน

ระบบมีผู้ใช้หลักสองกลุ่ม และมี Web Push provider เป็นระบบภายนอกที่รับคำขอส่งแจ้งเตือน ดังตารางที่ 3.1 และภาพที่ 3.2

ตารางที่ 3.1 ผู้ใช้และกรณีการใช้งานหลัก

ผู้ใช้	กรณีการใช้งานหลัก
ลูกค้า (ไม่ต้องสมัครบัญชี)	ดูเมนู ใส่ตะกร้า สร้างออเดอร์ ติดตามคิวและดูประวัติในเบราว์เซอร์ แก้ออเดอร์เมื่อ WAITING ยกเลิกเมื่อ WAITING/PREPARING เปิดหรือปิดการแจ้งเตือนของออเดอร์ รับแจ้งเตือนเมื่ออาหารพร้อม
พนักงาน (STAFF)	เข้า/ออกจากระบบ จัดการเมนูและรูป ดูรายการออเดอร์ เปลี่ยนสถานะคิว ยกเลิกคิวก่อน READY ดูผลส่งแจ้งเตือน สร้าง QR หน้าเมนู เปิด Swagger

ผู้ใช้งาน	กรณีการใช้งานหลัก
Web Push provider	รับคำขอส่งแจ้งเตือนจากระบบและส่งต่อไปยังเบราว์เซอร์



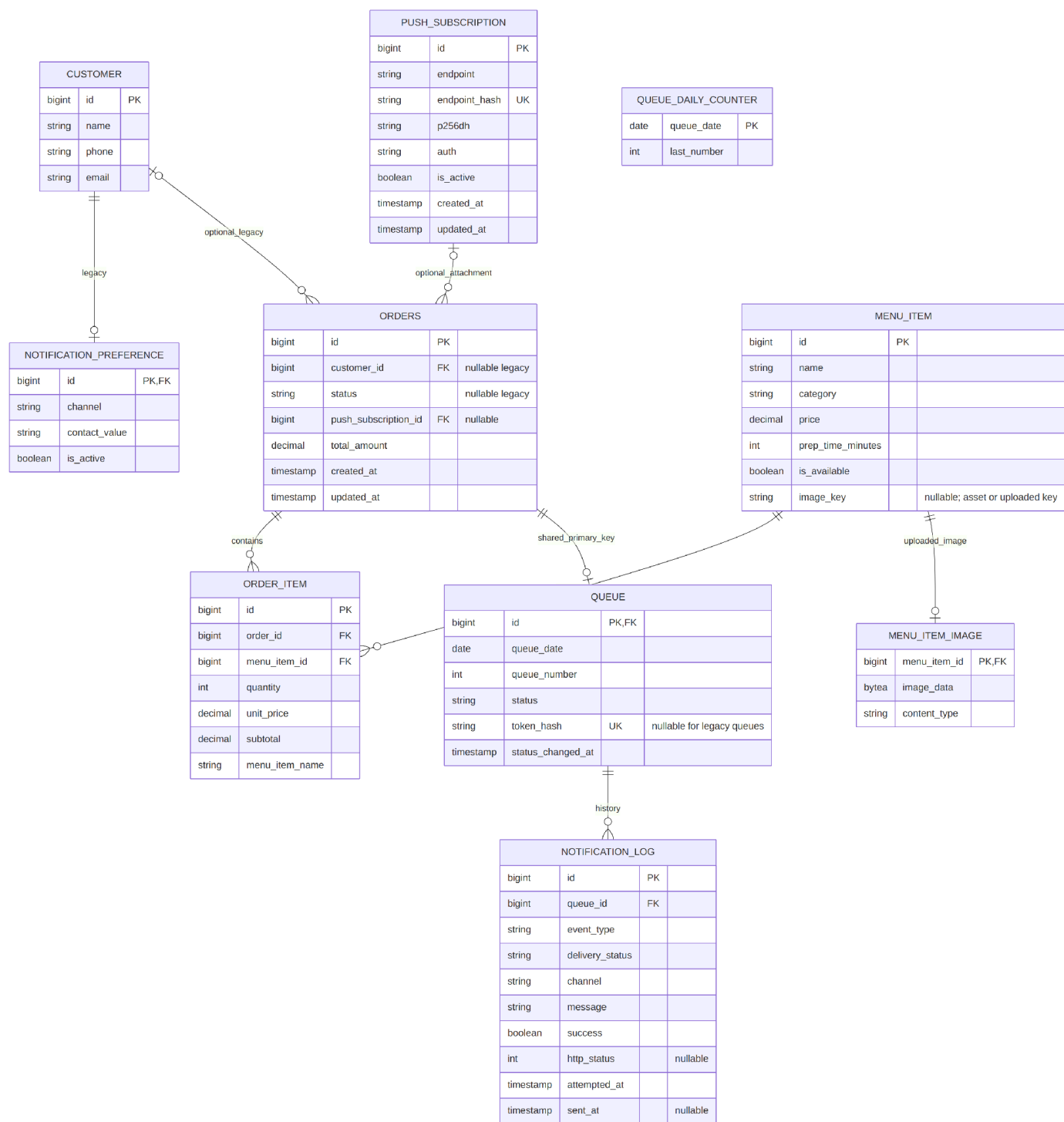
ภาพที่ 3.2 Use case ของระบบ

3.4 การออกแบบฐานข้อมูล

ฐานข้อมูล PostgreSQL จัดการโครงสร้างด้วย Flyway migration V1-V6 และตั้ง Hibernate เป็น ddl-auto: validate ตารางที่ใช้งานหลัก ได้แก่ orders, order_item, menu_item, menu_item_image, queue, queue_daily_counter, push_subscription และ notification_log ส่วน customer กับ notification_preference เป็นตาราง legacy ที่ flow ใหม่ไม่ต้องใช้ ดังตารางที่ 3.2 และภาพที่ 3.3 เวลาเก็บเป็น UTC ส่วนวันที่ของเลขคิวคำนวณตามเวลา Asia/Bangkok

ตารางที่ 3.2 ตารางฐานข้อมูลหลักและข้อจำกัดสำคัญ

ตาราง	ข้อมูลและข้อจำกัดสำคัญ
orders / order_item	เก็บออเดอร์และรายการ order_item เก็บ snapshot ชื่อและราคา ณ เวลาสั่ง และ unique(order_id, menu_item_id)
queue	ใช้ id ร่วมกับ orders เก็บ queue_date, queue_number, status, token_hash (unique) และ unique(queue_date, queue_number)
queue_daily_counter	ตัวนับเลขคิวรายวันตามวันที่ Asia/Bangkok ไม่ผูก FK กับคิวเพื่อไม่ให้เลขซ้ำเมื่อลบคิว
menu_item / menu_item_image	เมนูและรูป (JPEG/PNG ไม่เกิน 2 MiB เก็บเป็น bytea)
push_subscription	endpoint, endpoint_hash (unique), p256dh, auth และสถานะใช้งาน subscription หนึ่งรายการผูกได้หลายออเดอร์
notification_log	ผลส่งแจ้งเตือน มี partial unique index บน (queue_id, event_type) เฉพาะ READY เพื่อป้องกันส่งซ้ำ



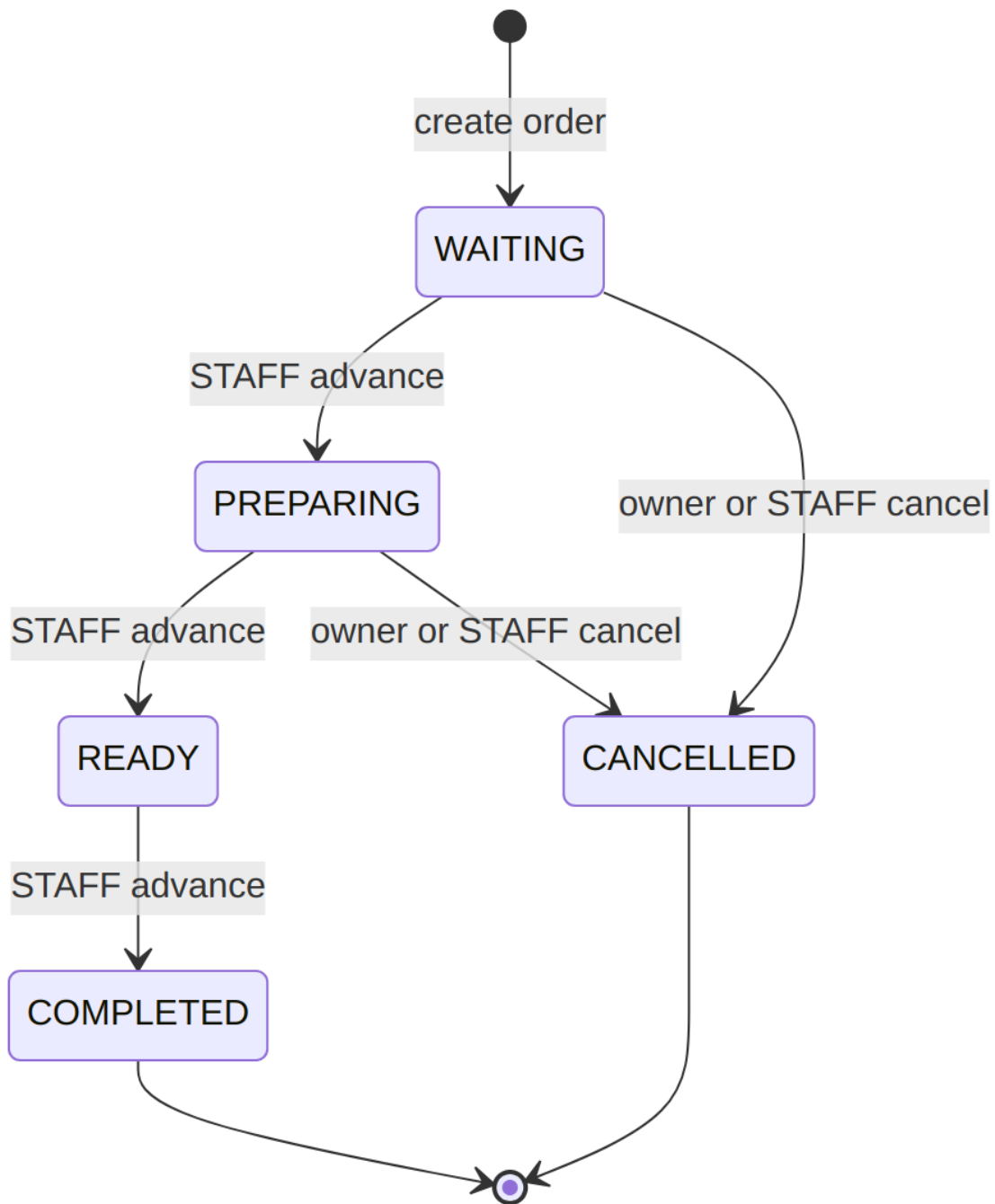
ภาพที่ 3.3 ER Diagram ของระบบ

3.5 การออกแบบสถานะของคิว

คิวมี 5 สถานะและเปลี่ยนได้ทางเดียวคือ WAITING → PREPARING → READY → COMPLETED โดยพนักงานเป็นผู้เลื่อนสถานะ การยกเลิกทำได้เฉพาะจาก WAITING หรือ PREPARING โดยเจ้าของออเดอร์หรือพนักงาน สถานะ READY ยกเลิกไม่ได้ ไม่มีการย้อนสถานะ และ COMPLETED/CANCELLED เป็นสถานะสิ้นสุด คำขอที่ละเมิดกฎจะตอบ 409 โดยไม่เปลี่ยนข้อมูล ดังตารางที่ 3.3 และภาพที่ 3.4

ตารางที่ 3.3 กฎการเปลี่ยนสถานะและการแก้ไขตามสถานะ

สถานะ	เลื่อน (STAFF)	ยกเลิก	แก้ไขรายการ	ลบบิล (STAFF)
WAITING	PREPARING	CANCELLED	ได้	ได้เมื่อไม่มี log
PREPARING	READY	CANCELLED	409	409
READY	COMPLETED	409	409	409
COMPLETED	409	409	409	409
CANCELLED	409	409	409	409



ภาพที่ 3.4 State diagram ของคิว

3.6 การออกแบบด้วย Design Pattern และหลักการ SOLID

3.6.1 Design Pattern กลุ่ม Behavioral

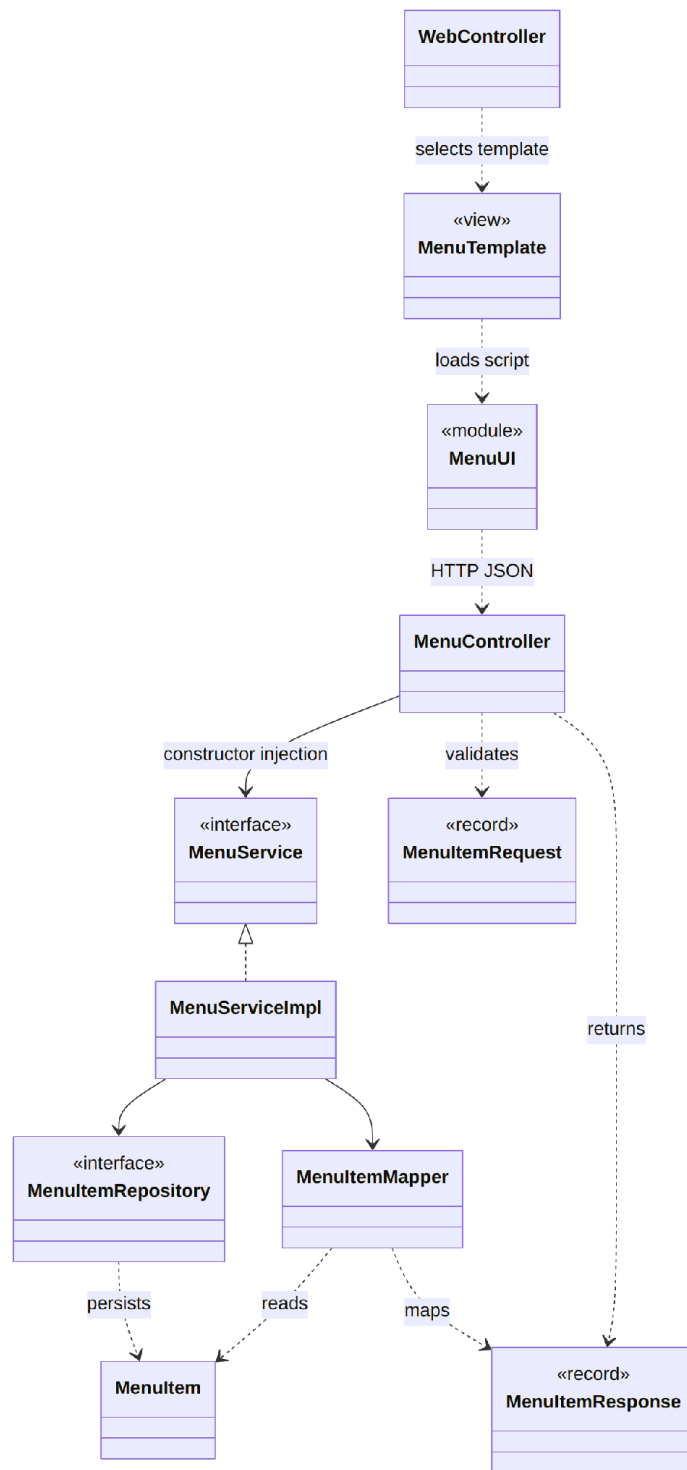
State กำหนดกฎการเคลื่อนและการยกเลิกของแต่ละสถานะไว้ในคลาสแยก ได้แก่ WaitingState, PreparingState, ReadyState, CompletedState และ CancelledState ซึ่งใช้ตามสัญญาของ QueueStateHandler โดย QueueServiceImpl รวบรวม handler ตามสถานะและตรวจว่าครบทุกสถานะ ส่วน QueueContext ถูกสร้างใหม่ทุก

operation เพื่อตรวจสอบว่าอยู่ใน transaction ที่เขียนได้ แล้วจึงเปลี่ยนสถานะ บันทึกเวลา และเผยแพร่ event วิธีนี้ทำให้ Controller ไม่ต้องมีเงื่อนไข if-else ของทุกสถานะ โครงสร้างแสดงในภาพที่ 3.6

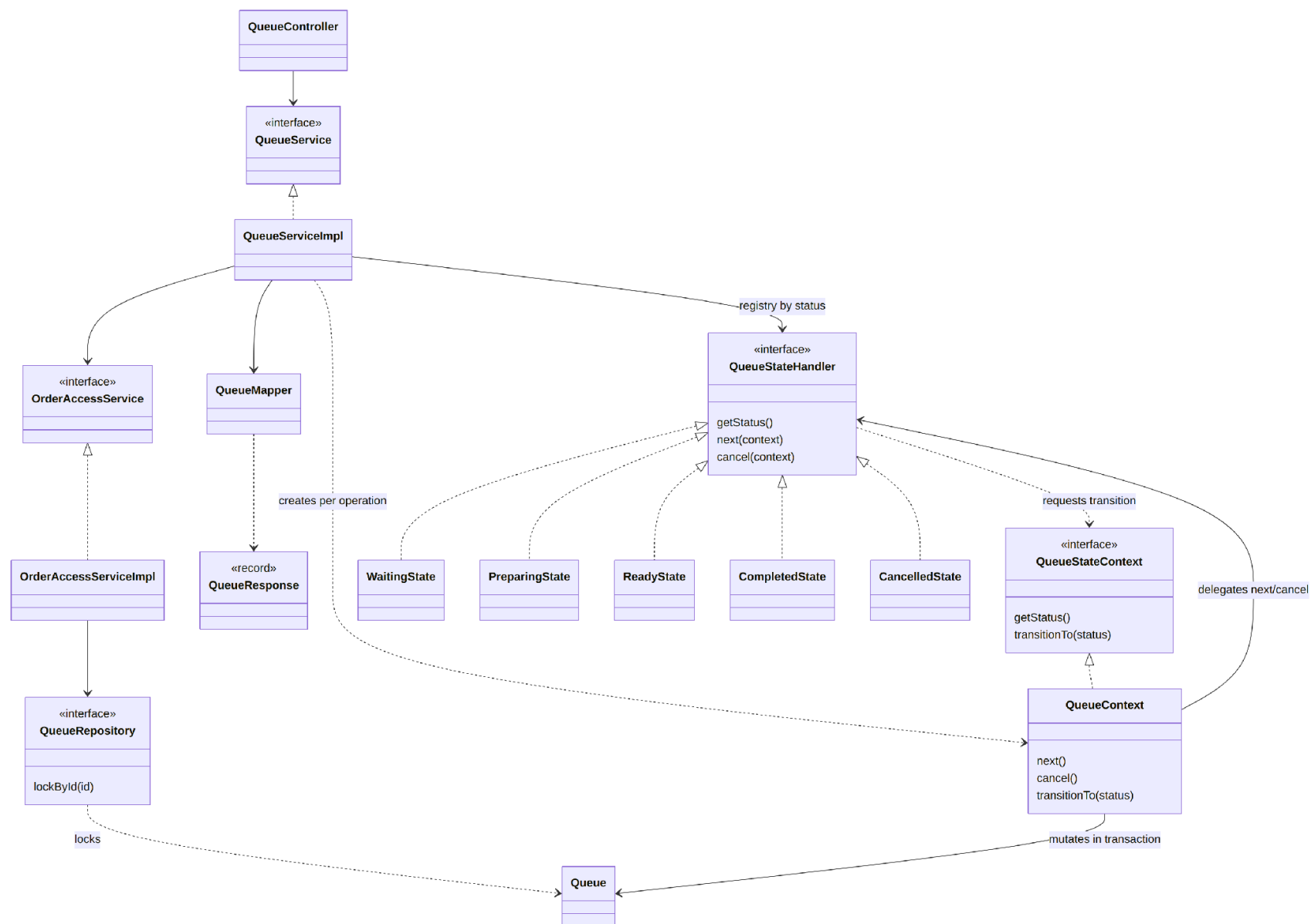
Observer ใช้แยกการเปลี่ยนสถานะออกจากการส่งแจ้งเตือน QueueContext เผยแพร่ QueueStatusChangedEvent ภายใน transaction และ ReadyNotificationObserver รับเหตุการณ์ด้วย @TransactionalEventListener(phase = AFTER_COMMIT) เฉพาะสถานะ READY ดังนั้นหาก transaction ล้มเหลวจะไม่มีคำสั่ง และหาก provider ล้มเหลวหลัง commit คิวก็ยังเปลี่ยนสถานะสำเร็จ นอกจากนี้ยังมี OrderSubscriptionAttachedEvent ร้องรับการแจ้งเตือนคำสั่งสมัครรับแจ้งเตือนหลังจากออเดอร์เป็น READY แล้ว โครงสร้างแสดงในภาพที่ 3.7

Strategy กำหนดช่องทางส่งผ่านอินเทอร์เฟซ NotificationStrategy โดย ConsoleNotificationStrategy ใช้สำหรับ preview และ PushNotificationStrategy ส่งจริงผ่าน BrowserWebPushSender (เข้ารหัสและลงลายเซ็น VAPID) การเลือกทำตามค่า notification.mode (console หรือ webpush) NotificationDeliveryServiceImpl จึงไม่ต้องรู้รายละเอียดการเข้ารหัส หรือ HTTP โครงสร้างแสดงในภาพที่ 3.8

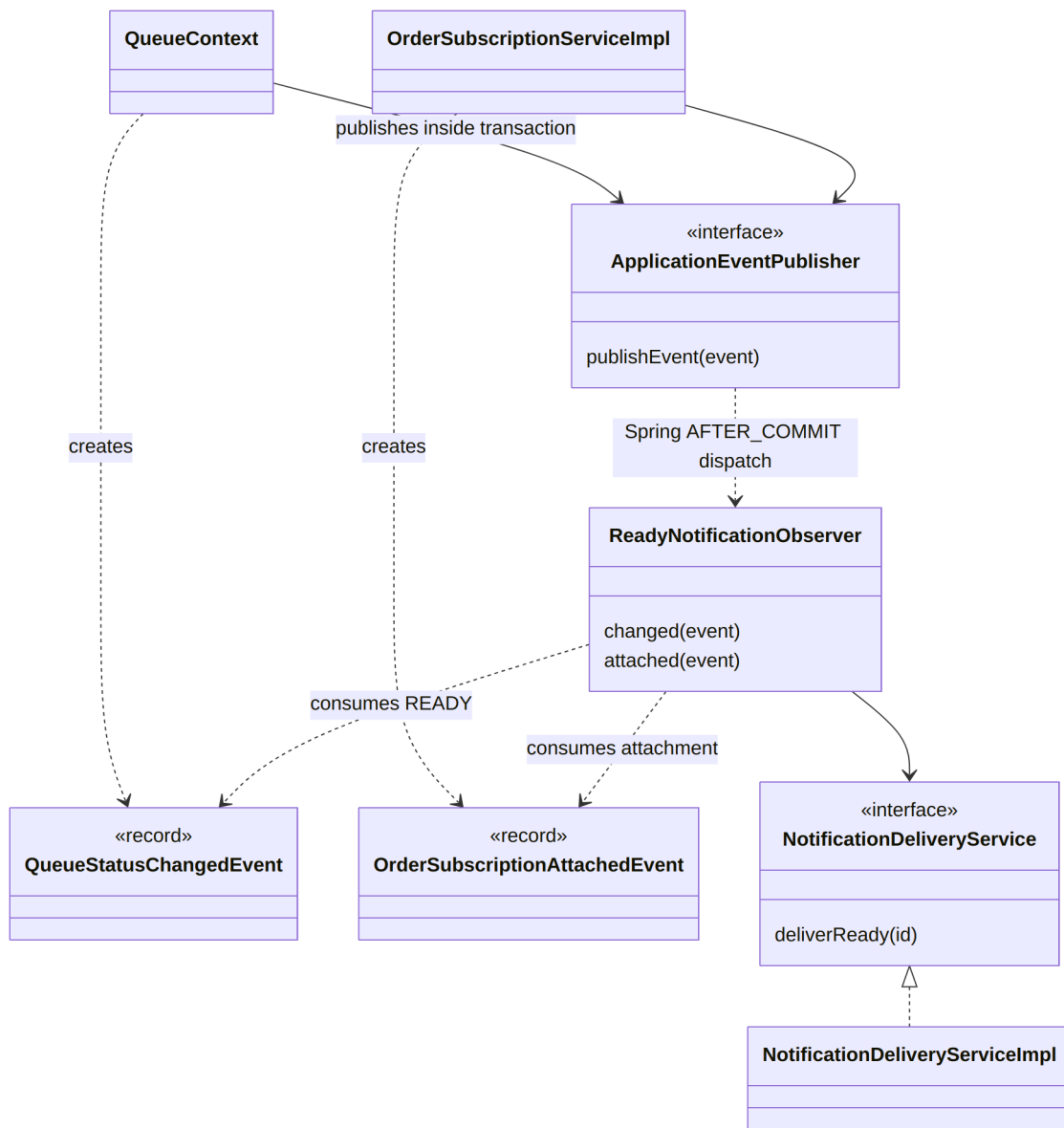
ส่วนสถาปัตยกรรมแบบชั้นและ MVC แสดงในภาพที่ 3.5 ซึ่งแสดงตัวอย่างเส้นทางของเมนูตั้งแต่หน้าเว็บ Controller Service Repository จนถึง DTO และ Mapper



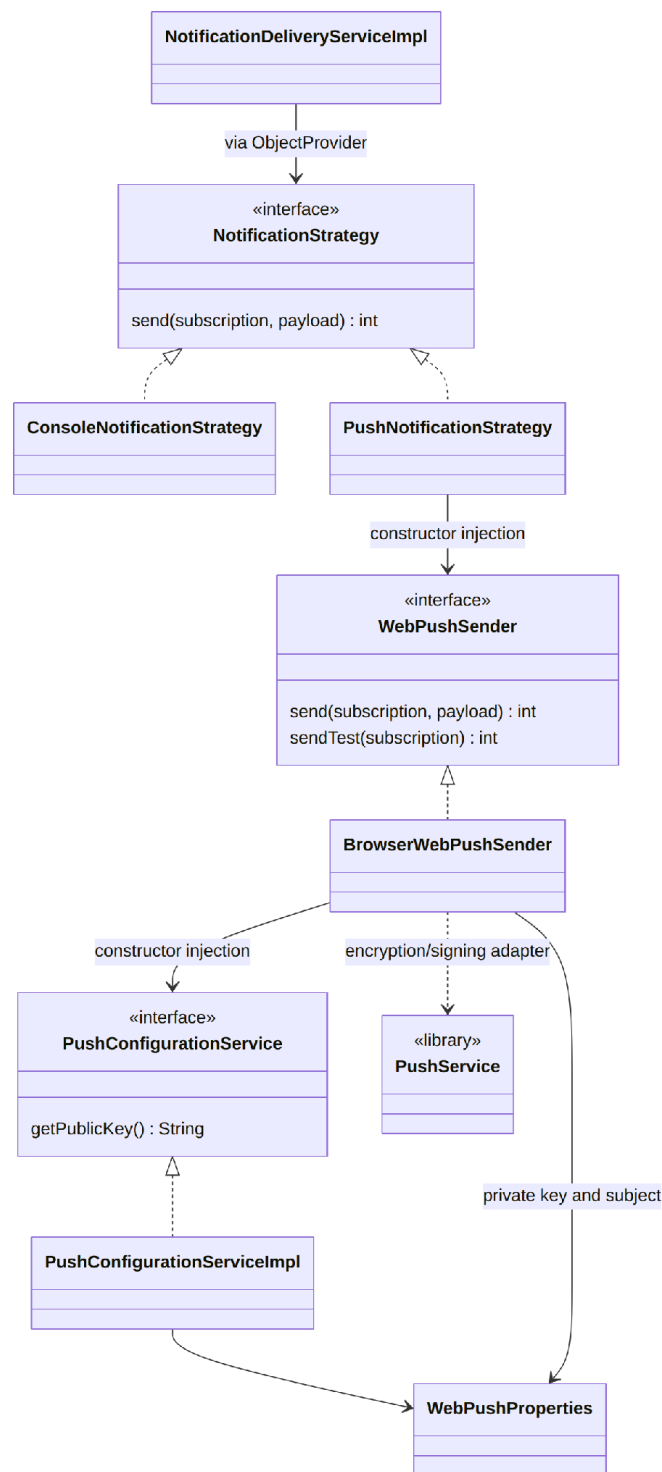
ภาพที่ 3.5 Class diagram ของ Layered Architecture และ MVC



ภาพที่ 3.6 Class diagram ของ State Pattern



ภาพที่ 3.7 Class diagram ของ Observer Pattern



ภาพที่ 3.8 Class diagram ของ Strategy Pattern

3.6.2 หลักการ SOLID

การออกแบบอ้างอิงหลักการ SOLID โดยสรุปหลักฐานในโค้ดตามตารางที่ 3.4 ทั้งนี้การมีอินเทอร์เฟซหรือชื่อคลาสเพียงอย่างเดียวไม่ถือว่าทุกคลาสผ่านหลักการ และการเพิ่มสถานะหรือช่องทางแจ้งเตือนใหม่ยังต้องปรับ enum, handler registry, configuration และ log metadata ให้ครอบคลุม

ตารางที่ 3.4 หลักการ SOLID และตัวอย่างในระบบ

หลักการ	การนำมาใช้	ตัวอย่างคลาส
SRP	Controller จัดการ HTTP/validation, Service จัด use case และ transaction, Repository ติดต่อฐานข้อมูล, Mapper สร้าง DTO และส่ง Push แยก observer, delivery และ sender	OrderController, OrderServiceImpl, OrderMapper, ReadyNotificationObserver, BrowserWebPushSender
OCP	QueueContext เรียก handler ผ่าน interface และ delivery เรียก Strategy ผ่าน interface จึงแยกพฤติกรรมแต่ละสถานะและแต่ละช่องทางออกจากกัน	QueueContext, QueueServiceImpl, NotificationStrategy
LSP	State handler ทุกตัวทำตามสัญญา next/cancel และ NotificationStrategy ระบุว่า 0 คือ preview ส่วนค่าอื่นเป็น HTTP status ของ provider	QueueStateHandler, ReadyState, CompletedState
ISP	แยกการสร้าง/hash token, ตรวจสอบสิทธิ์เจ้าของ, จัดเลขคิว, สมัคร subscription และส่งแจ้งเตือนเป็นคนละ interface	QueueTokenGenerator, OrderAccessService, QueueNumberService, NotificationDeliveryService
DIP	Controller และ Service รับ service ports ผ่าน constructor โดย Spring เป็นผู้เชื่อม implementation	QueueServiceImpl, ReadyNotificationObserver, BrowserWebPushSender

3.7 การออกแบบความปลอดภัย

ลูกค้าไม่มีบัญชี ระบบจึงออก **owner token** ให้แต่ละออเดอร์ตอนสร้าง สร้างด้วย SecureRandom 32 ไบต์ เข้ารหัส Base64 URL-safe และเก็บในฐานข้อมูลเฉพาะ hash แบบ SHA-256 (`queue.token_hash`) เปรียบเทียบกับ `MessageDigest.isEqual` และส่ง token ดิบกลับเฉพาะตอนสร้าง ลูกค้าส่ง token ผ่าน header `X-Queue-Token` พนักงานล็อกอินด้วย username/password จากค่าตั้งค่าสภาพแวดล้อม รหัสผ่านแฮชด้วย BCrypt ต้องยาวอย่างน้อย 12 ตัวอักษร และหากไม่ตั้งรหัสจะล็อกอินไม่ได้ ระบบปิด HTTP Basic ใช้ session และบังคับ CSRF กับทุกคำขอที่เปลี่ยนข้อมูล สิทธิ์ของแต่ละกลุ่มเส้นทางเป็นไปตามตารางที่ 3.5

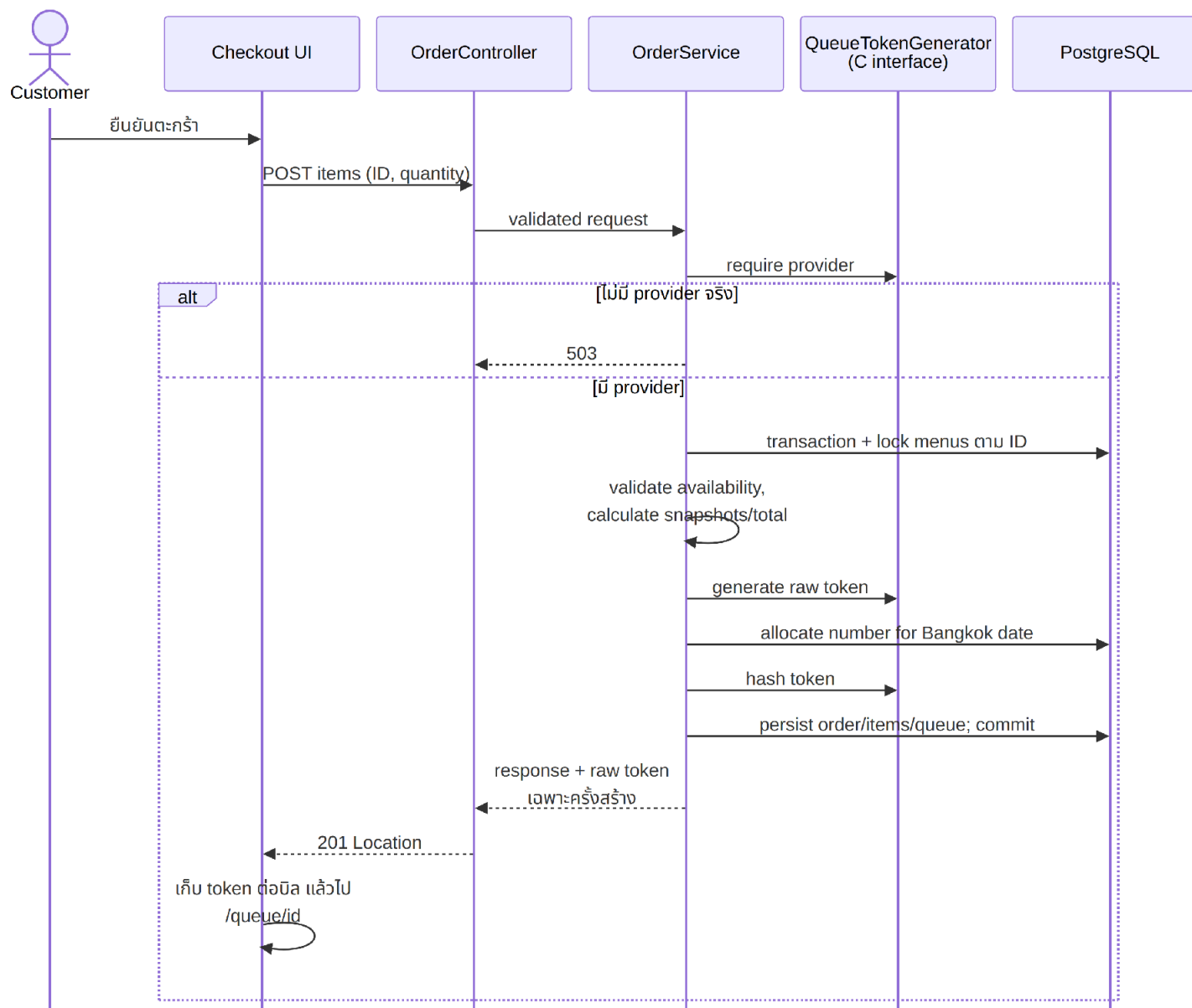
ตารางที่ 3.5 สิทธิ์การเข้าถึงตามกลุ่มเส้นทาง

กลุ่มเส้นทาง	ผู้เข้าถึงได้
ดูเมนู, public key, CSRF token	ทุกคน
สร้างออเดอร์	ทุกคน (ต้องผ่าน CSRF และ validation)
อ่าน/แก้ไข/ยกเลิกออเดอร์, สมัคร-ยกเลิก subscription	เจ้าของ (token) หรือ STAFF
เลื่อนสถานะคิว, ดูรายการออเดอร์, ดู log, จัดการเมนู, QR, Swagger	STAFF

เพื่อป้องกันการแก้ไขพร้อมกัน ระบบล็อกแถวคิวแบบ pessimistic write ก่อนตรวจสอบสิทธิ์และเปลี่ยนสถานะ ข้อผิดพลาดตอบเป็น JSON รูปแบบเดียวกันโดยไม่เปิดเผย stack trace หรือ SQL

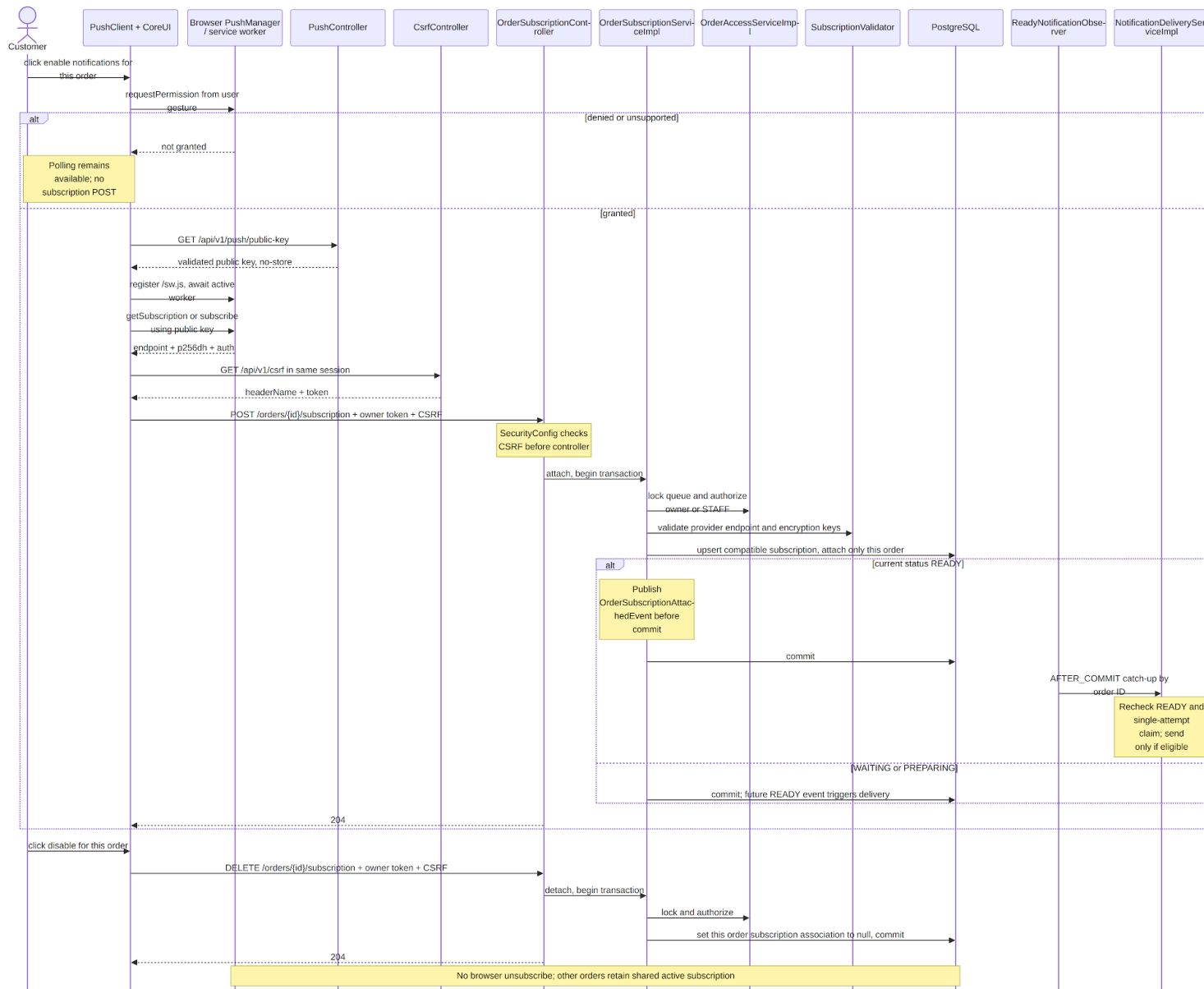
3.8 กระบวนการทำงานหลัก

การสร้างออเดอร์ เซิร์ฟเวอร์ตรวจสอบเมนูและราคา สร้าง snapshot และยอดรวม ออก token แล้วจัดสรรเลขคิวรายวันภายใน transaction เดียว หากเกิดข้อผิดพลาดจะ rollback รวมถึงตัวนับเลขคิว ดังภาพที่ 3.9



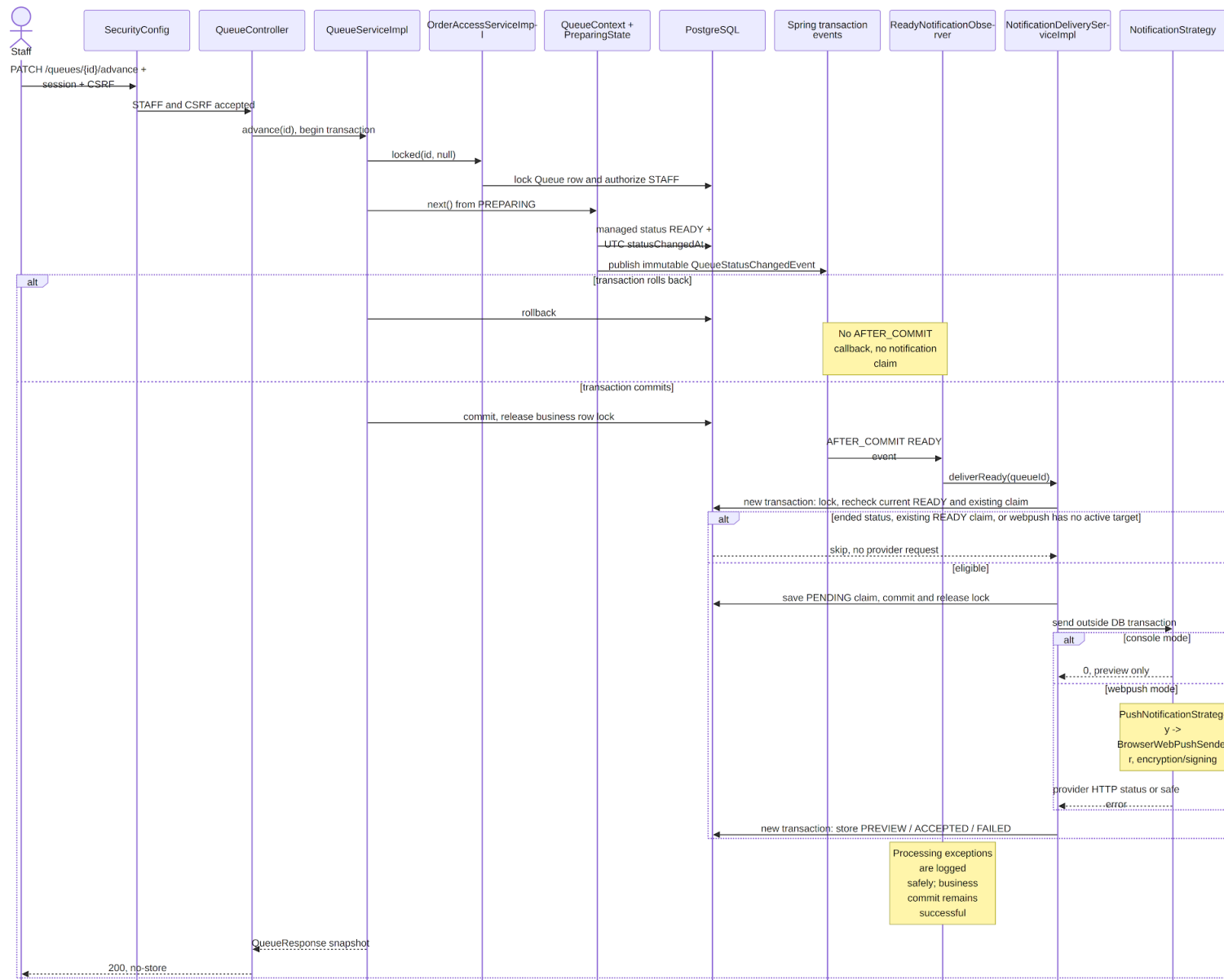
ภาพที่ 3.9 Sequence diagram การสร้างออเดอร์

การสมัครรับแจ้งเตือน เบราร์เซอร์ขอสิทธิ์ ลงทะเบียน Service Worker และสมัคร PushManager ด้วย public key จากนั้นส่ง endpoint และ key ไปผูกกับออเดอร์ด้วย owner token และ CSRF ระบบตรวจ endpoint/keys ก่อนบันทึก หากสมัครตอนออเดอร์เป็น READY แล้วจะมีการ catch-up ส่งหนึ่งครั้งหากยังมีสิทธิ์ ลูกค้านำไม่เปิดการแจ้งเตือนยังติดตามคิวผ่าน polling ได้ ดังภาพที่ 3.10



ภาพที่ 3.10 Sequence diagram การสมัครและยกเลิก Push ของออเดอร์

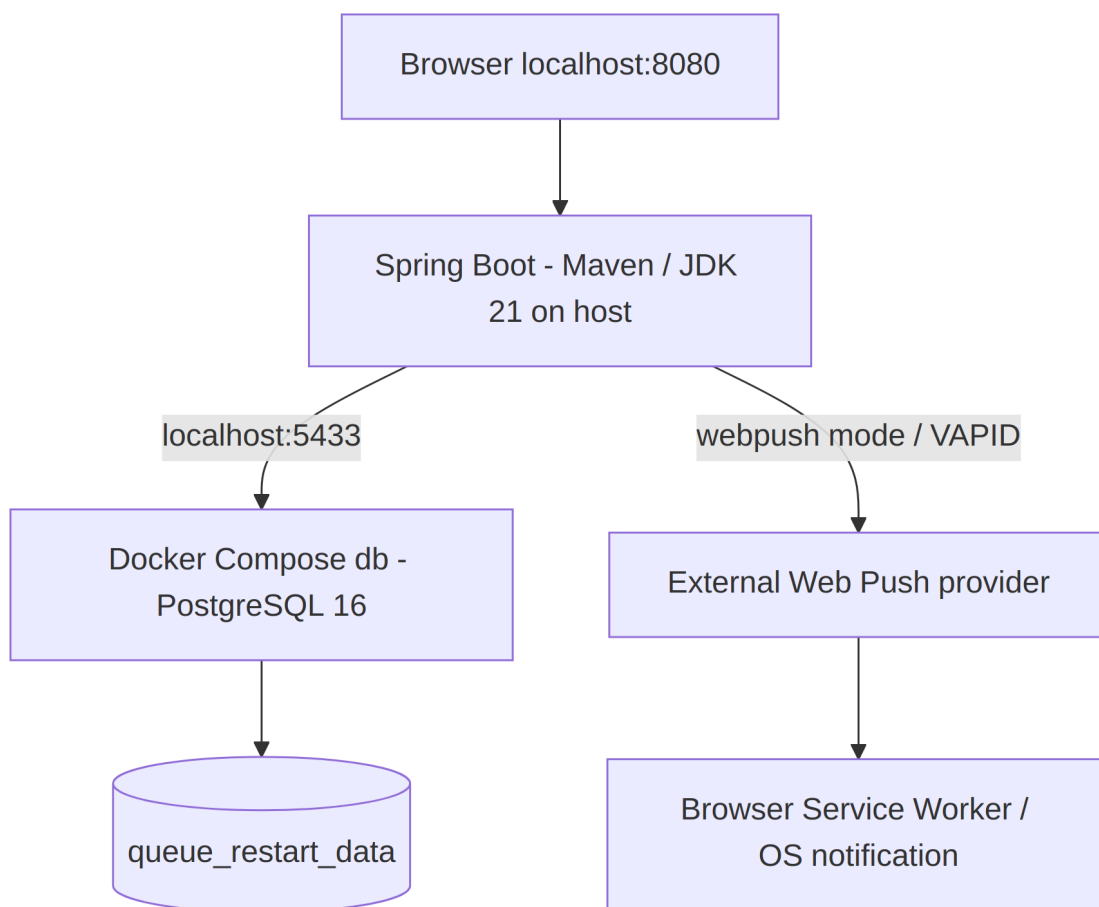
การแจ้งเตือนเมื่อ READY หลัง commit Observer เรียก delivery service ซึ่งตรวจสอบสถานะปัจจุบันและ claim ที่มีอยู่ จากนั้นบันทึก claim สถานะ PENDING ใน transaction แยกแล้วจึงส่งจริงนอก transaction และบันทึกผลเป็น PREVIEW (console), ACCEPTED (provider รับคำขอ) หรือ FAILED โดยไม่มี automatic retry ข้อความที่ส่งคือ “อาหารคาว (เลขคาว) พร้อมแล้วเชิญรับอาหารได้เลย” ดังภาพที่ 3.11



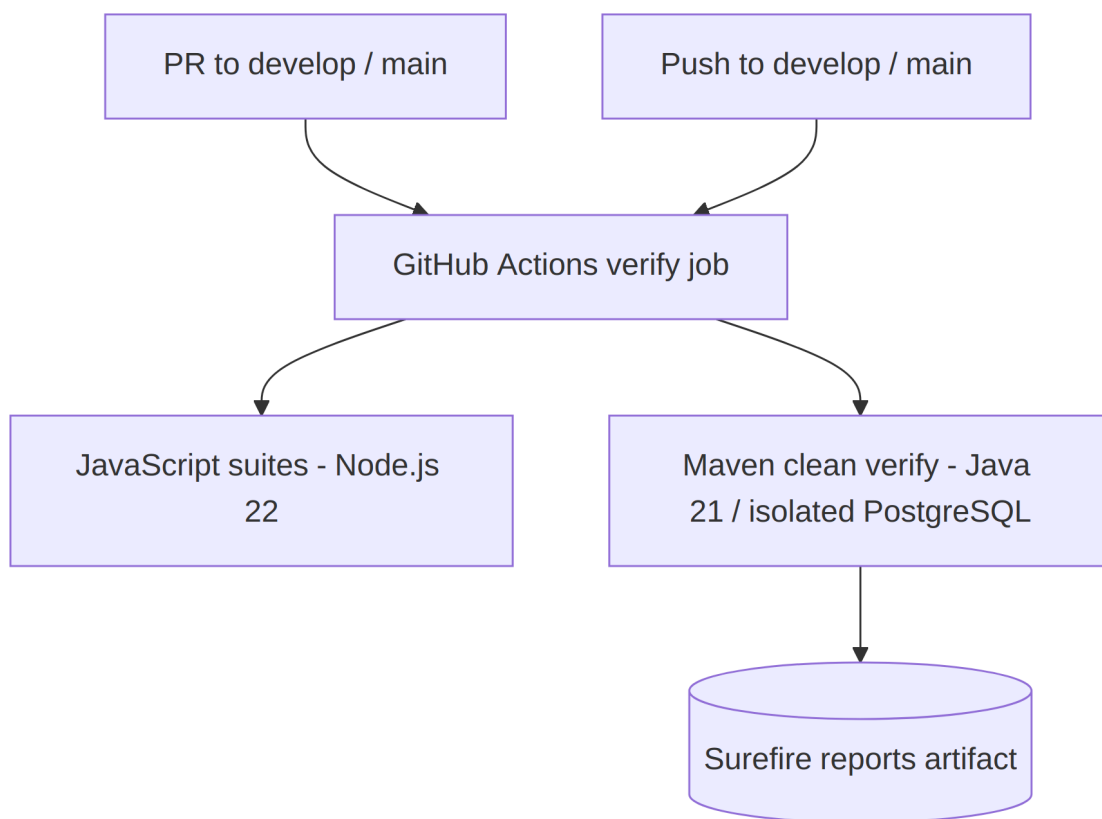
ภาพที่ 3.11 Sequence diagram การแจ้งเตือนหลัง commit เมื่อคิวเป็น READY

3.9 การติดตั้งและการใช้งาน

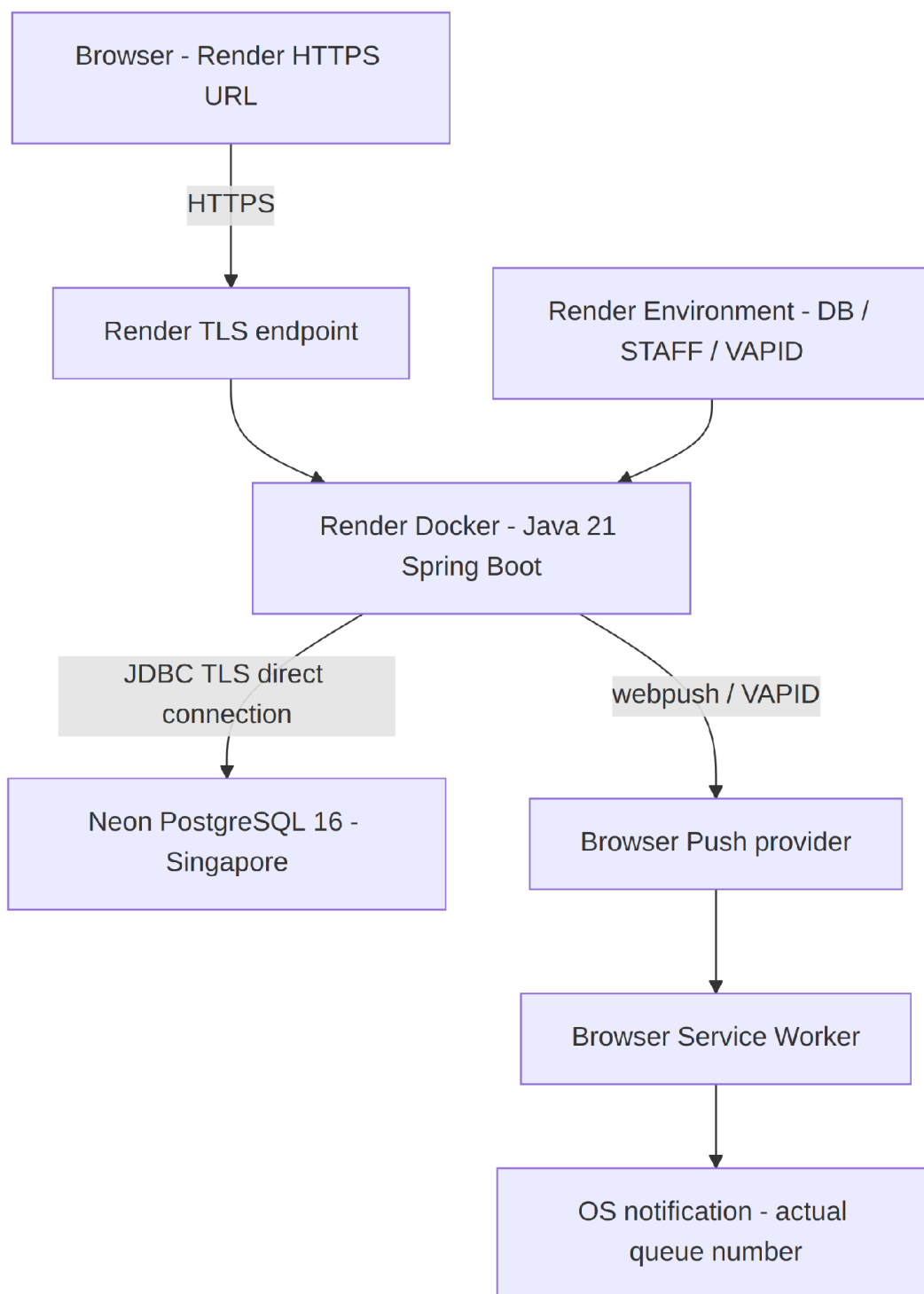
ระบบรองรับการรันในเครื่องด้วย Maven ร่วมกับ PostgreSQL บน Docker Compose (ภาพที่ 3.12) การรันทั้งแอปด้วย Docker Compose และการ deploy บนคลาวด์ โดยใช้ Render (Docker, Singapore) เป็น web service และ Neon PostgreSQL 16 เป็นฐานข้อมูลผ่าน HTTPS (ภาพที่ 3.14) ค่า DB รหัส STAFF และ VAPID key ตั้งใน environment ไม่เก็บใน repository ส่วน CI ใช้ GitHub Actions รัน JavaScript tests และ `mvn clean verify` กับ PostgreSQL แบบแยก (ภาพที่ 3.13) โดยยังไม่มี deploy job อัตโนมัติ



ภาพที่ 3.12 การรันในเครื่องที่ใช้ทดสอบ



ภาพที่ 3.13 CI บน GitHub Actions



ภาพที่ 3.14 การ deploy บน Render และ Neon PostgreSQL

3.10 วิธีการทดสอบ

การทดสอบแบ่งเป็นสามส่วน ส่วนแรกคือชุดทดสอบ Java รันด้วย `mvn clean verify` ครอบคลุม unit, MVC/Security และ PostgreSQL integration tests บนฐานข้อมูลแบบ embedded ที่แยกจากฐานข้อมูลของแอป ส่วนที่สองคือชุดทดสอบ JavaScript ด้วย Node.js

สำหรับ UI และ Service Worker และส่วนที่สามคือการทดสอบด้วยตนเองบนระบบที่ deploy โดยสั่ง
อาหารและตรวจการแจ้งเตือนจริง ผลการทดสอบแสดงในบทที่ 4

บทที่ 4 ผลการดำเนินงาน

4.1 สภาพแวดล้อมและเครื่องมือ

พัฒนาด้วย JDK 21 (compile เป้าหมาย Java 17) Maven 3.9 Spring Boot 3.2.5 PostgreSQL 16 และ Node.js สำหรับชุดทดสอบ JavaScript ทีมทำงานผ่าน GitHub โดยมี branch develop สำหรับรวมงานและรวมผ่าน Pull Request ที่มีสมาชิกอีกคนรีวิว

4.2 ผลการพัฒนาระบบ (วัตถุประสงค์ข้อ 1)

ระบบที่พัฒนาเสร็จครอบคลุมทั้งฝั่งลูกค้าและฝั่งพนักงาน รวมถึงการจัดสรรเลขคิวและการแจ้งเตือน ดังตารางที่ 4.1 และภาพประกอบหน้าจอของระบบ

ตารางที่ 4.1 ผลการพัฒนาตามฟังก์ชัน

ฟังก์ชัน	ผลที่พัฒนาเสร็จ
ฝั่งลูกค้า	หน้าเมนู ตะกร้า สร้างออเดอร์ ประวัติและติดตามคิว แก้/ยกเลิกตามกฎ
ฝั่งพนักงาน	ล็อกอิน จัดการเมนูและรูป เลื่อนสถานะ ดูรายการและ log สร้าง QR
เลขคิว	เลขรายวันตามเวลา Bangkok ไม่ซ้ำเมื่อสั่งพร้อมกัน และไม่ถูกใช้เมื่อ rollback
แจ้งเตือน	Subscription, Observer หลัง commit, Strategy console/webpush, log, กันซ้ำ
เอกสาร API	Swagger/OpenAPI เป็ดเฉพาะ STAFF

[วางภาพหน้าจอ: หน้าเมนูและตะกร้าสินค้า]

ภาพที่ 4.1 หน้าเมนูและตะกร้าสินค้า

[วางภาพหน้าจอ: หน้าติดตามคิวของลูกค้า]

ภาพที่ 4.2 หน้าติดตามคิวของลูกค้า

[วางภาพหน้าจอ: หน้าจอพนักงานสำหรับเลื่อนสถานะคิว]

ภาพที่ 4.3 หน้าจอพนักงานสำหรับเลื่อนสถานะคิว

[วางภาพหน้าจอ: การแจ้งเตือนเมื่ออาหารพร้อมบนอุปกรณ์]

ภาพที่ 4.4 การแจ้งเตือนเมื่ออาหารพร้อมบนอุปกรณ์

4.3 ผลการประยุกต์ Design Pattern (วัตถุประสงค์ข้อ 2)

ระบบประยุกต์ใช้ State, Observer และ Strategy จริงในโค้ดพร้อมชุดทดสอบที่เกี่ยวข้อง ดังตารางที่ 4.2 ทั้งนี้ enum QueueStatus เพียงอย่างเดียวไม่ใช่ State Pattern หลักฐานของ pattern คือ handler interface, implementation และการมอบหมายงานผ่าน Context ที่มีอยู่จริง

ตารางที่ 4.2 Design Pattern ที่ใช้และชุดทดสอบที่เกี่ยวข้อง

Pattern	คลาสหลัก	ชุดทดสอบที่เกี่ยวข้อง
State	QueueStateHandler, QueueContext, WaitingState, PreparingState, ReadyState, CompletedState, CancelledState	WaitingPreparingStateTest, ReadyTerminalStateTest, QueueContextTest, QueueServiceTest, QueueStateIntegrationTest
Observer	QueueStatusChangedEvent, ReadyNotificationObserver, OrderSubscriptionAttachedEvent	QueueTransitionEventTest, ReadyNotificationObserverTest, ReadyNotificationIntegrationTest
Strategy	NotificationStrategy, ConsoleNotificationStrategy, PushNotificationStrategy, NotificationDeliveryServiceImpl	NotificationStrategyTest, NotificationDeliveryTest, PushDeliveryIntegrationTest

4.4 ผลการออกแบบความปลอดภัย (วัตถุประสงค์ข้อ 3)

ระบบนำกลไกความปลอดภัยมาใช้ตามตารางที่ 4.3 โดยมีชุดทดสอบที่ตรวจทั้งในระดับ service และระดับ API ที่ผ่าน security filter จริง

ตารางที่ 4.3 กลไกความปลอดภัยและชุดทดสอบที่เกี่ยวข้อง

กลไก	การทำงานในระบบ	ชุดทดสอบที่เกี่ยวข้อง
Owner token	สร้างด้วย SecureRandom 32 ไบต์ เก็บเฉพาะ SHA-256 hash ตรวจสอบผ่าน header X-Queue-Token	OrderAccessServiceTest, OrderAccessIntegrationTest
STAFF login	รหัสผ่านแฮชด้วย BCrypt ใช้ session ปิด HTTP Basic ไม่มี default credential	CsrfSecurityTest, OpenApiSecurityTest

กลไก	การทำงานในระบบ	ชุดทดสอบที่เกี่ยวข้อง
CSRF	ทุกคำขอที่เปลี่ยนข้อมูลต้องมี CSRF token ของ session ที่ขอมา	CsrfSecurityTest
การแก้ไขพร้อมกัน	ล็อกแถวคิวแบบ pessimistic write ก่อนตรวจสอบสิทธิ์ และเปลี่ยนสถานะ	OrderAccessTransactionTest, QueueStateIntegrationTest

4.5 ผลการทดสอบอัตโนมัติ (วัตถุประสงค์ข้อ 4)

ตามรายงานของทีม mvn clean verify ผ่าน 285 tests ไม่มี failure, error หรือ skipped เมื่อวันที่ 10 ตุลาคม 2026 และชุดทดสอบ JavaScript ผ่าน 14 ไฟล์ ชุดทดสอบ Java ใช้ PostgreSQL แบบ embedded แยกจากฐานข้อมูลแอป ขอบเขตของชุดทดสอบเป็นไปตามตารางที่ 4.4

ตารางที่ 4.4 ขอบเขตของชุดทดสอบอัตโนมัติ

ขอบเขต	ตัวอย่างชุดทดสอบ
Migration และฐานข้อมูล	MigrationTest, AnonymousMigrationTest, DailyQueueMigrationTest, MenuPhotoMigrationTest, EntityMappingTest
เลขคิวรายวันและการสั่งพร้อมกัน	DailyQueueAllocationTest, DailyQueueIntegrationTest
เมนูและรูป	MenuControllerTest, MenuCrudPersistenceTest, MenuImageControllerTest, MenuImageIntegrationTest
ออเดอร์และสิทธิ์เจ้าของ	OrderControllerTest, OrderCrudPersistenceTest, OrderAccessServiceTest, OrderAccessIntegrationTest
การแจ้งเตือน	NotificationDeliveryTest, NotificationDeliveryPersistenceTest, NotificationLogServiceTest, NotificationStrategyTest
UI และ Service Worker (JavaScript)	basket, checkout, history, queue-tracking, push-client, service-worker, staff-queue, staff-menu และอื่น ๆ รวม 14 ไฟล์

ชุดทดสอบการแจ้งเตือนครอบคลุมการไม่ส่งเมื่อ rollback การตรวจสอบสถานะปัจจุบันก่อนส่ง การกัน claim ซ้ำ การเข้ารหัสและลงลายเซ็นของ Web Push และกรณี provider ตอบ 410/429/500 ที่บันทึกเป็น FAILED

4.6 ผลการทดสอบบนระบบที่ deploy

ผู้ทดสอบ (กานติทัต นามสุดตา) บันทึกผลเมื่อ 10 ตุลาคม 2026 ใช้ revision c07292e ของ branch develop ดังตารางที่ 4.5

ตารางที่ 4.5 ผลการทดสอบบน Neon และ Render

ขั้นตอน	ผล
Maven ในเครื่องเชื่อม Neon และ Flyway สร้าง schema (6 migrations)	ผ่าน
แอปในเครื่องเริ่มทำงานกับ Neon	ผ่าน
Render build/deploy และแอปเริ่มทำงาน	ผ่าน
สร้างออดเอร์และรับ Push บน URL Render	ผ่านตามผู้ทดสอบ และเลขคิวในข้อความถูกต้อง

นอกจากนี้ทีมยืนยันว่ารูปเมนูคงเดิมหลังเปลี่ยนชื่อหรือหมวดหมู่ และ Swagger เรียกเมนู ID 1 ได้ HTTP 200

4.7 สิ่งที่ยังไม่ยืนยัน

ผลข้างต้นไม่ใช่ acceptance ครบทุกข้อ รายการตรวจรับ (acceptance checklist) ใน repository ยังไม่ได้ทำเครื่องหมายครบ เช่น owner isolation, CSRF และ terminal state บน URL จริง การอัปโหลดภาพ ประวัติ และ QR บนโทรศัพท์ นอกจากนี้ สถานะ ACCEPTED หมายถึง provider รับคำขอ ไม่ยืนยันว่าอุปกรณ์แสดงแจ้งเตือนทุกครั้ง และผลทดสอบบนอุปกรณ์จริงยังไม่ได้บันทึกบนอุปกรณ์/เบราว์เซอร์ [ถ้าทดสอบเพิ่มก่อนส่ง ให้เติมผลที่นี่]

บทที่ 5 สรุปผล อภิปรายผลและข้อเสนอแนะ

5.1 สรุปผลการดำเนินงาน

โครงการพัฒนาระบบ Food Queue Notifier ที่ให้ลูกค้าสั่งอาหาร รับเลขคิวรายวัน ติดตามสถานะ และรับแจ้งเตือนผ่าน Browser Push เมื่ออาหารพร้อม ขณะที่พนักงานจัดการเมนูและเลื่อนสถานะคิวได้ สอดคล้องกับวัตถุประสงค์ข้อ 1 ระบบประยุกต์ State, Observer และ Strategy ร่วมกับสถาปัตยกรรมแบบชั้น ตามวัตถุประสงค์ข้อ 2 และออกแบบความปลอดภัยด้วย owner token, BCrypt, CSRF และการล็อกแถวเพื่อป้องกันการแก้ไขพร้อมกัน ตามวัตถุประสงค์ข้อ 3 ผลทดสอบอัตโนมัติผ่าน 285 tests และ JavaScript 14 ไฟล์ และระบบ deploy บน Render/Neon ได้พร้อมผ่านการทดสอบสั่งอาหารและรับแจ้งเตือนจริงหนึ่งรอบ ตามวัตถุประสงค์ข้อ 4

5.2 อภิปรายผล

การให้ Observer รับเหตุการณ์หลัง transaction commit ทำให้ระบบไม่ส่งแจ้งเตือนเมื่อการเปลี่ยนสถานะล้มเหลว และความล้มเหลวของ provider ไม่ทำให้คิวที่ commit แล้วดูเหมือนล้มเหลว แต่ผลที่ตามมาคือ Observer เป็น event ภายในแอป จึงไม่มีการ replay หากโปรเซสหยุดหลัง commit การแยกผลการส่งเป็น PREVIEW, ACCEPTED และ FAILED ทำให้รายงานผลตรงกับหลักฐาน เพราะ ACCEPTED เป็นเพียงการที่ provider รับคำขอ ไม่ใช่การยืนยันว่าอุปกรณ์แสดงแจ้งเตือน

การใช้ owner token แทนบัญชีลูกค้าลดขั้นตอนการใช้งานและเหมาะกับการสั่งอาหารที่ต้องการความเร็ว แต่ทำให้ประวัติผูกกับเบราว์เซอร์และไม่มีการกู้ token การใช้ State Pattern ทำให้กฎของแต่ละสถานะแยกชัดเจนและทดสอบได้ แต่การเพิ่มสถานะใหม่ยังต้องแก้ enum, handler registry และชุดทดสอบให้ครบ

5.3 ปัญหาและอุปสรรค

ระหว่างพัฒนาพบปัญหา เช่น การทดสอบ migration รอบแรกผิดที่ชื่อตารางในคำสั่งทดสอบ (แก้ที่ test โดยไม่แก้ migration ที่ใช้แล้ว) Flyway 9.22.3 แจ้งเตือนว่า PostgreSQL 16.15 ใหม่กว่าเวอร์ชันที่ทดสอบ แต่ migration ยังทำงานสำเร็จ subscription ที่สมัครจาก localhost ใช้กับ origin ของ Render ไม่ได้ ต้องสมัครใหม่ และ Compose ปัจจุบันยังไม่ส่งค่า STAFF/VAPID ให้ container ต้องตั้งเพิ่มก่อนทดสอบแบบครบ

5.4 ข้อจำกัดของระบบ

Observer เป็น event ภายในแอป ไม่ใช่ message broker จึงไม่มี outbox หรือ replay และไม่มี automatic retry หากโปรเซสหยุดหลัง commit คำขอเลื่อนสถานะซ้ำสองคำขอที่ถูกสิทธิ์อาจเลื่อนสองขึ้นต่อเนื่อง เพราะ API ยังไม่มี expected-state หรือ idempotency key ลูกค้าไม่มีบัญชีและไม่มีการกู้ token ประวัติจึงผูกกับเบราว์เซอร์ การเลือกช่องทางแจ้งเตือนเป็นค่าตั้งค่าของแอป และ Render Free อาจพักเมื่อไม่มีการใช้งานทำให้เริ่มช้า

5.5 ข้อเสนอแนะ

ข้อเสนอแนะต่อการนำไปใช้ประโยชน์ ระบบเหมาะกับร้านอาหารหรือเครื่องดื่มที่มีลูกค้านำสินค้าหน้าร้านและต้องการลดความแออัด ก่อนนำไปใช้จริงควร merge develop เข้า main แล้วตั้ง Render ให้ deploy จาก main พร้อมบันทึก SHA และทำ acceptance ให้ครบ

ข้อเสนอแนะต่อการพัฒนาต่อยอด ควรเพิ่ม transactional outbox หรือกลไก retry สำหรับการส่งแจ้งเตือน เพิ่ม expected-state หรือ idempotency key ให้การเลื่อนสถานะ ทำ acceptance บนอุปกรณ์และเบราว์เซอร์จริงหลายรุ่นพร้อมบันทึกหลักฐาน เพิ่มช่องทางแจ้งเตือนอื่น (enum NotificationChannel มี LINE และ SMS อยู่แล้ว) และพิจารณาบัญชีลูกค้าหรือกลไกประวัติ

เอกสารอ้างอิง

- Gamma, E., Helm, R., Johnson, R., & Vlissides, J. (1994). **Design patterns: Elements of reusable object-oriented software**. Addison-Wesley.
- Martin, R. C. (2003). **Agile software development, principles, patterns, and practices**. Prentice Hall.
- OWASP. (n.d.). **Cross-Site Request Forgery Prevention Cheat Sheet**. OWASP Cheat Sheet Series.
- Thomson, M., Damaggio, E., & Raymor, B. (Eds.). (2016). **Generic event delivery using HTTP push** (RFC 8030). Internet Engineering Task Force.
- Thomson, M., & Beverloo, P. (2017). **Voluntary application server identification (VAPID) for web push** (RFC 8292). Internet Engineering Task Force.
- เอกสารประกอบโครงการใน repository food-queue-notifier (branch develop): README, design-patterns, solid-analysis, security-and-state, state-test-report, push-test-report, deployment-test-report และแผนภาพใน doc/diagrams.
- [ตรวจสอบและปรับปรุงแบบการอ้างอิงตามที่อาจารย์กำหนด และเพิ่มงานที่เกี่ยวข้องจากหัวข้อ 2.8]

ภาคผนวก

ภาคผนวก ก โครงสร้างโฟลเดอร์ของโปรเจกต์

ตารางที่ ก.1 โครงสร้างโฟลเดอร์ของโปรเจกต์

ตำแหน่ง	เนื้อหา
code/src/main/java	controller, service/interface, mapper, DTO, entity และ repository
code/src/main/resources	configuration, Flyway migration (V1–V6), template และ static assets
test/java	unit, MVC/Security และ PostgreSQL integration tests
test/js, test/fixtures	UI/Service Worker tests และข้อมูลจำลองเฉพาะการทดสอบ
doc	แบบจำลอง รายงานทดสอบ acceptance และคู่มือส่งต่อ
tools/GenerateVapidKeys.java	เครื่องมือสร้าง VAPID key pair สำหรับการตั้งค่าในเครื่อง

ภาคผนวก ข ภาพหน้าจอและหลักฐานการทดสอบเพิ่มเติม

[แนบภาพหน้าจอและหลักฐานการทดสอบบนอุปกรณ์จริง โดยปิดบัง token รหัสผ่านและข้อมูลลับ]